
Курс: Рекомендательные системы

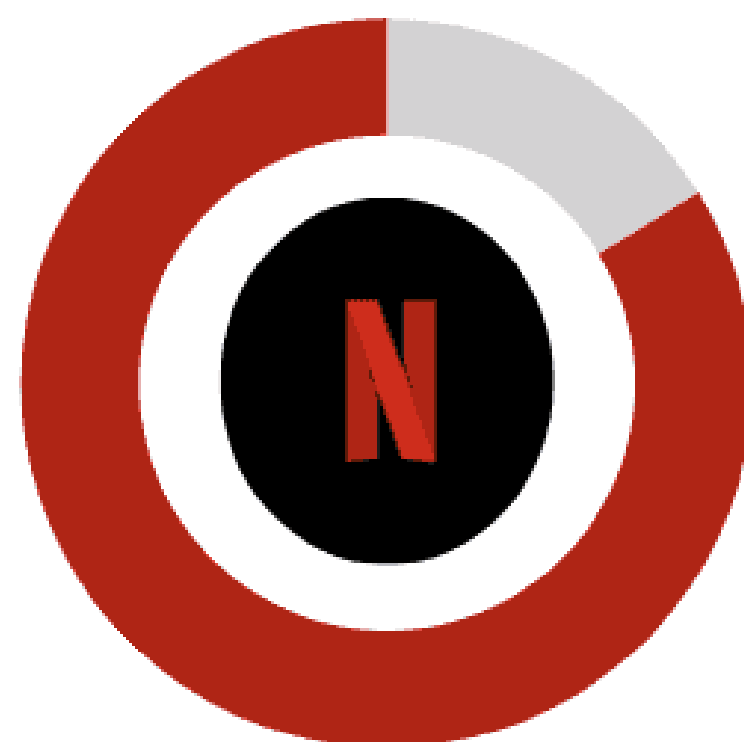
Занятие 5:

Контентные и контекстные ПОДХОДЫ

Красильников Денис
11.02.25

Контент и контекст

Идея: Дать дополнительную полезную информацию по взаимодействиям пользователя, при помощи описания объекта, пользователя или условий, при которых было совершено взаимодействие.



80%

of content consumed on
Netflix is due to
recommendations.



60%

of video clicks on
Youtube's homepage
are attributed to
recommendations



35%

of its revenue is generated
by its recommendation
engine

Контент и контекст



Идея: Дать дополнительную полезную информацию по взаимодействиям пользователя, при помощи описания объекта, пользователя или условий, при которых было совершено взаимодействие.

Контент –

Контекст –

Контент и контекст



Идея: Дать дополнительную полезную информацию по взаимодействиям пользователя, при помощи описания объекта, пользователя или условий, при которых было совершено взаимодействие.

Контент – описательное свойство объекта/пользователя, которое, как правило, не изменяется во времени или делает это очень редко.

Контекст –

Контент и контекст



Идея: Дать дополнительную полезную информацию по взаимодействиям пользователя, при помощи описания объекта, пользователя или условий, при которых было совершено взаимодействие.

Контент – описательное свойство объекта/пользователя, которое, как правило, не изменяется во времени или делает это очень редко.

Контекст – внешние условия или быстро меняющиеся признаки.

Контент и контекст



Контекст:

- Время
- Геолокация
- Девайс получения рекомендаций
- Количество денег на счету
- Поисковый запрос недавний
- Межличностные отношения
- Внешние факторы
- Погода на улице

Контент:

- Описание
- Категории товара
- Пол пользователя
- Имя пользователя
- Режиссер фильма
- Жанр фильма
- Актер главной роли
- Знак зодиака пользователя

Как добавить контекст/контетн в модели?

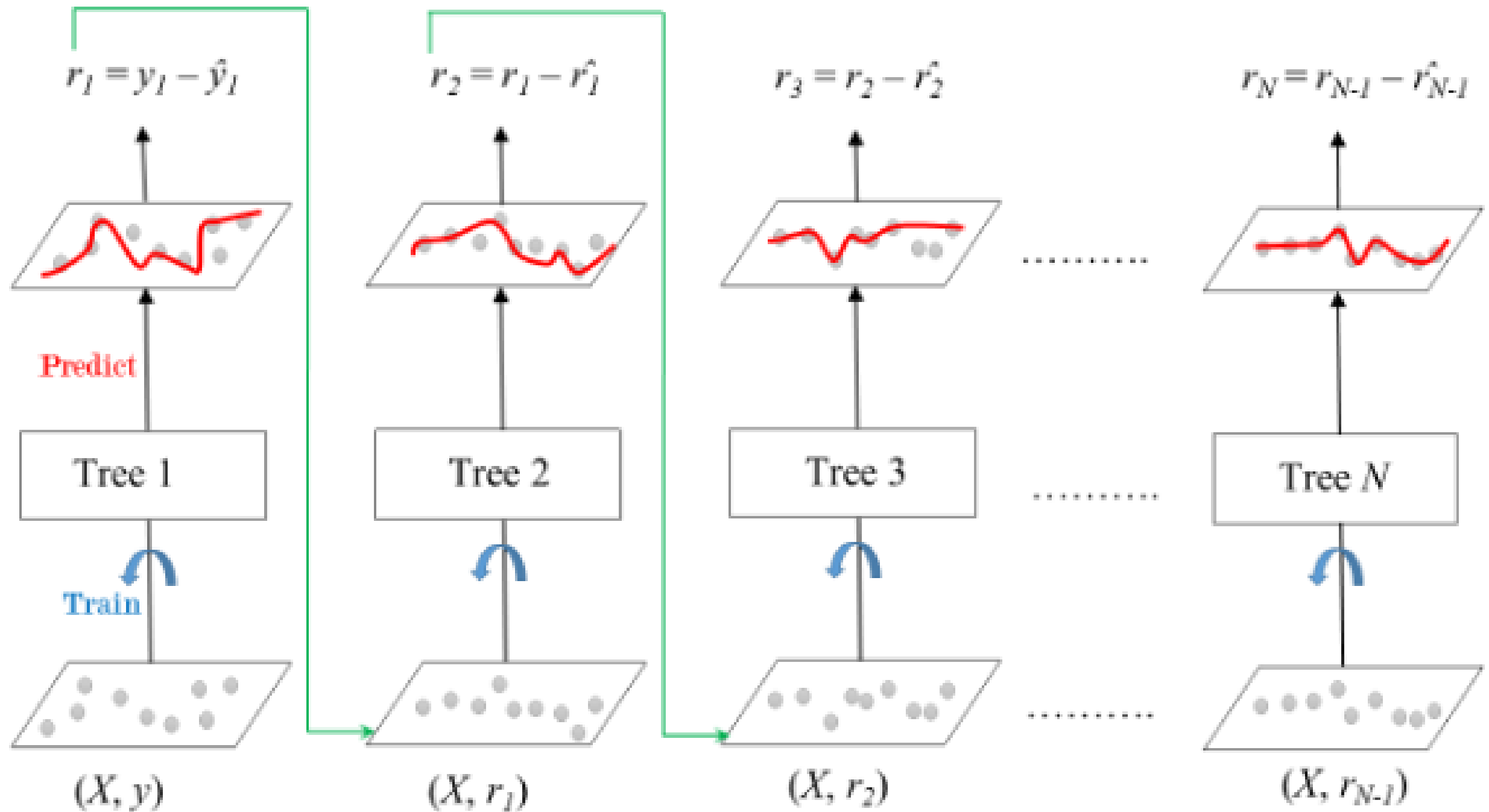


Gradient boosting

 **LightGBM**

 **CatBoost**

dmlc
XGBoost



Gradient boosting



Logloss

$$-\sum_{i=1}^N w_i (c_i \log(p_i) + (1 - c_i) \log(1 - p_i))$$

$$\sum_{i=1}^N w_i$$

PairLogit

$$-\sum_{p,n \in Pairs} w_{pn} \left(\log\left(\frac{1}{1 + e^{-(a_p - a_n)}}\right) \right)$$

$$\sum_{p,n \in Pairs} w_{pn}$$

YetiRank

$$\mathbb{L} = -\sum_{(i,j)} w_{ij} \log \frac{e^{x_i}}{e^{x_i} + e^{x_j}},$$

Deep Structured Semantic Model



Learning Deep Structured Semantic Models for Web Search using Clickthrough Data

Po-Sen Huang

University of Illinois at Urbana-Champaign
405 N Mathews Ave. Urbana, IL 61801 USA
huang146@illinois.edu

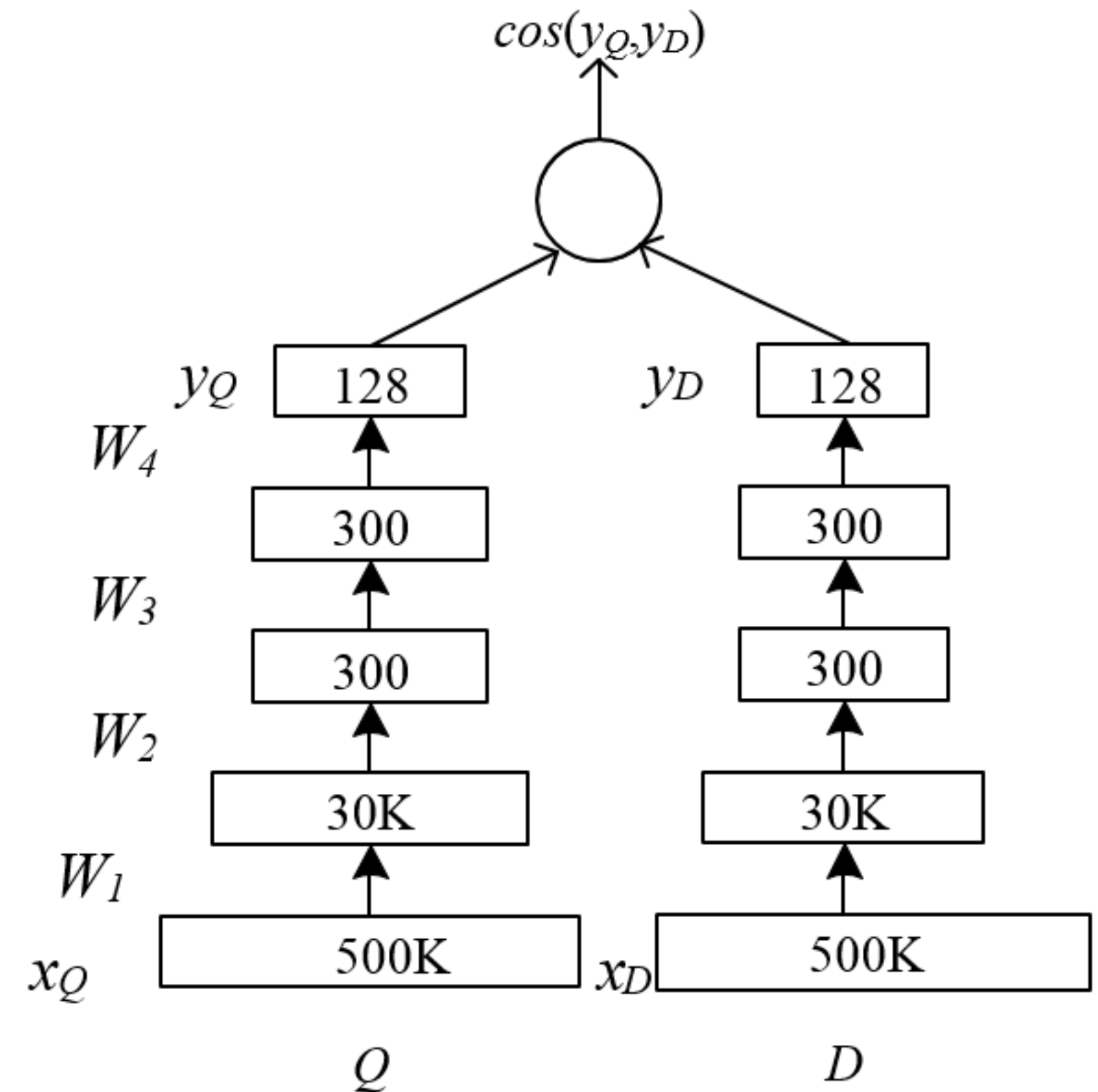
Xiaodong He, Jianfeng Gao, Li Deng,

Alex Acero, Larry Heck

Microsoft Research, Redmond, WA 98052 USA
{xiaohe, jfgao, deng, alexac, lheck}@microsoft.com

ABSTRACT

Latent semantic models, such as LSA, intend to map a query to its relevant documents at the semantic level where keyword-based matching often fails. In this study we strive to develop a series of new latent semantic models with a deep structure that project queries and documents into a common low-dimensional space where the relevance of a document given a query is readily computed as the distance between them. The proposed deep structured semantic models are discriminatively trained by maximizing the conditional likelihood of the clicked documents given a query using the clickthrough data. To make our models applicable to large-scale Web search applications, we also use a technique called word hashing, which is shown to effectively scale up our semantic models to handle large vocabularies which are common in such tasks. The new models are evaluated on a Web document ranking task using a real-world data set. Results show that our best model significantly outperforms other latent semantic models, which were considered state-of-the-art in the performance prior to the work presented in this paper.



Huang PS, He X, Gao J, Deng L, Acero A, Heck L. Learning deep structured semantic models for web search using clickthrough data. In Proceedings of the 22nd ACM international conference on Information & Knowledge Management 2013 Oct 27 (pp. 2333-2338).

Deep Structured Semantic Model

Posterior probability
computed by softmax

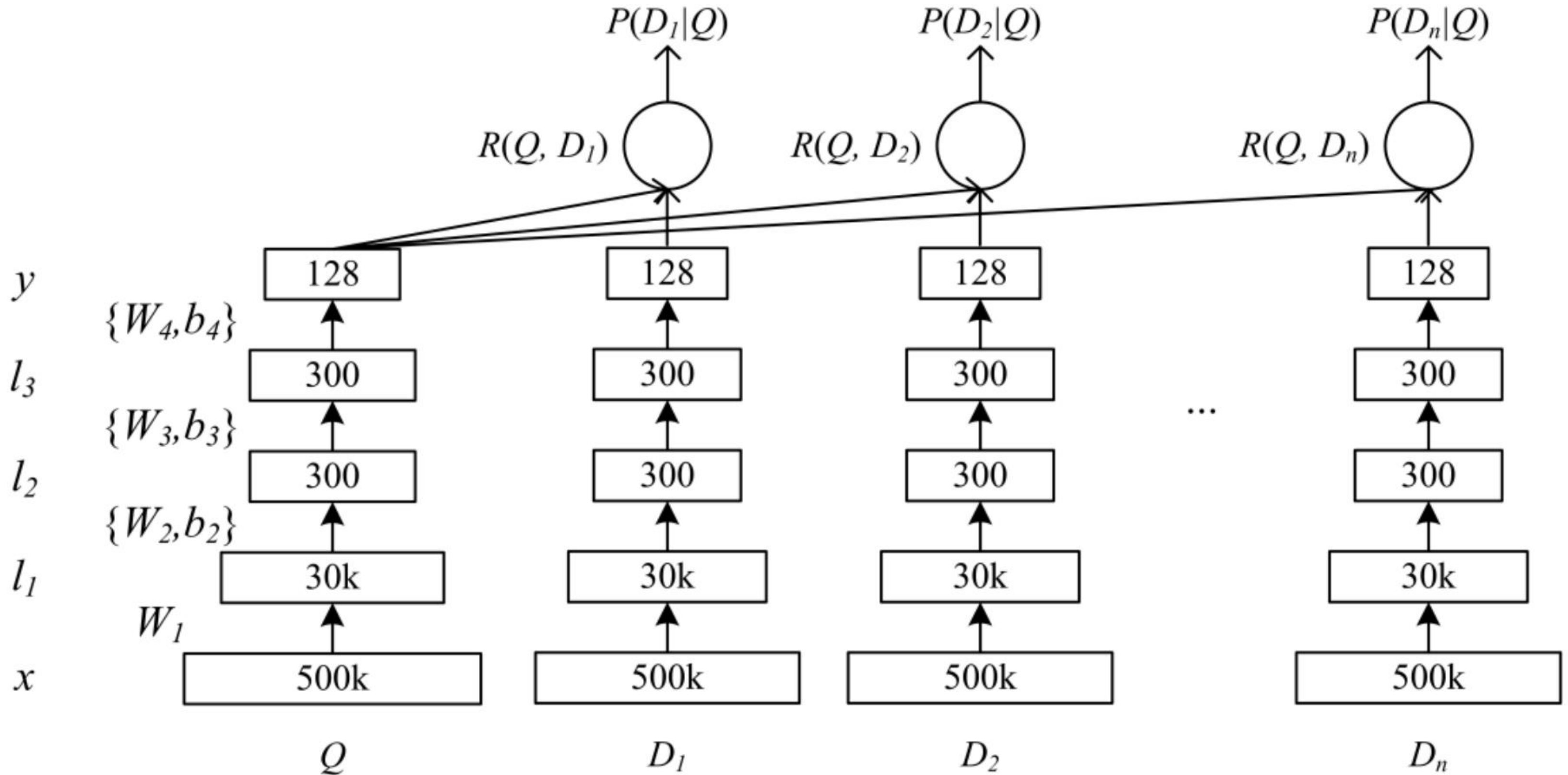
Relevance measured
by cosine similarity

Semantic feature

Multi-layer non-
linear projection

Word Hashing

Term Vector



Deep Structured Semantic Model



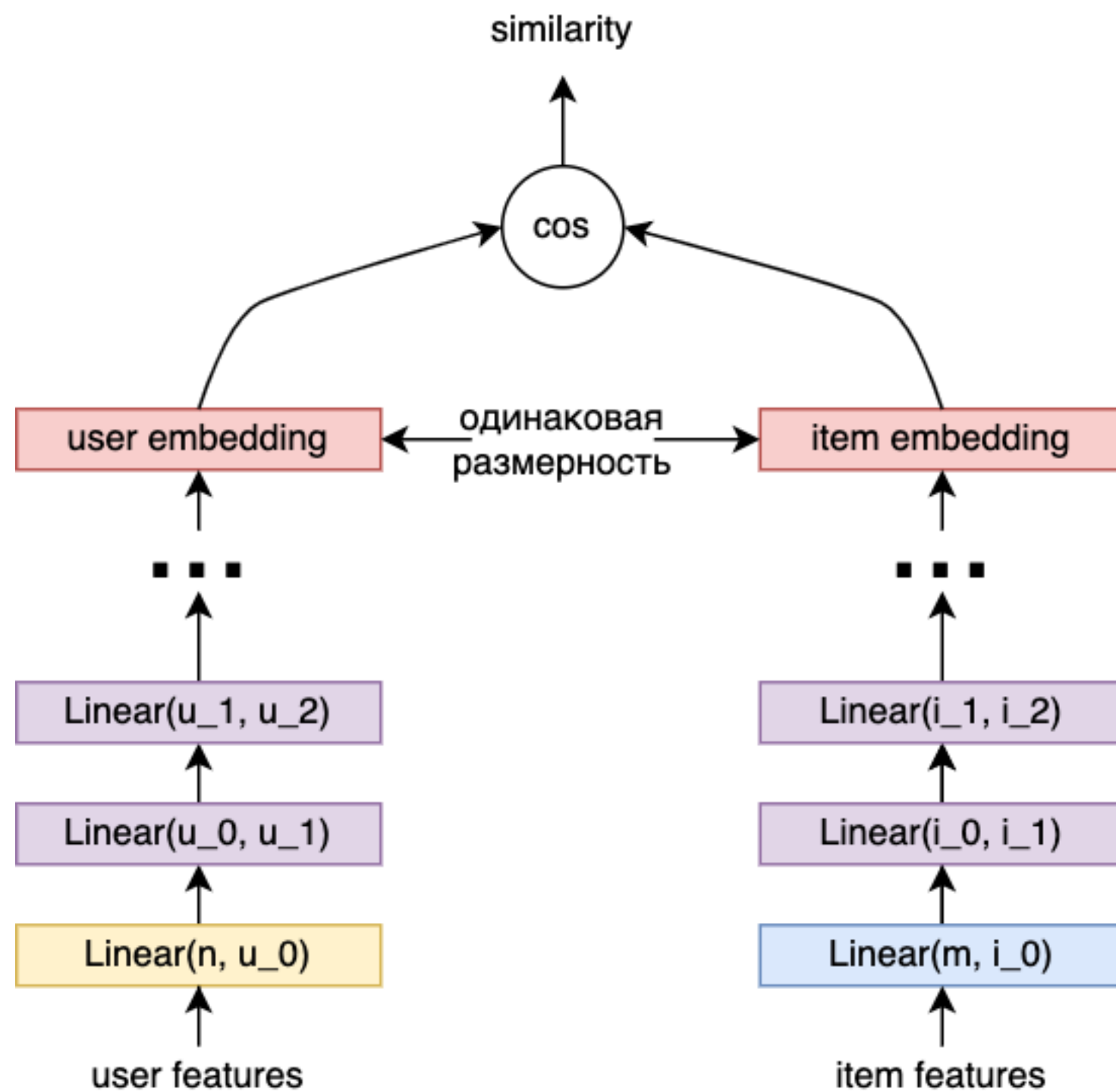
$$R(Q, D) = \text{cosine}(y_Q, y_D) = \frac{y_Q^T y_D}{\|y_Q\| \|y_D\|}$$

$$P(D|Q) = \frac{\exp(\gamma R(Q, D))}{\sum_{D' \in D} \exp(\gamma R(Q, D'))} \quad \gamma - \text{smoothing factor}$$

$$L(\Lambda) = -\log \prod_{(Q, D^+)} P(D^+|Q)$$

D^+ – позитивный пример
 D^- – негативные примеры
 $D = D^+ \cup D^-$

Deep Structured Semantic Model

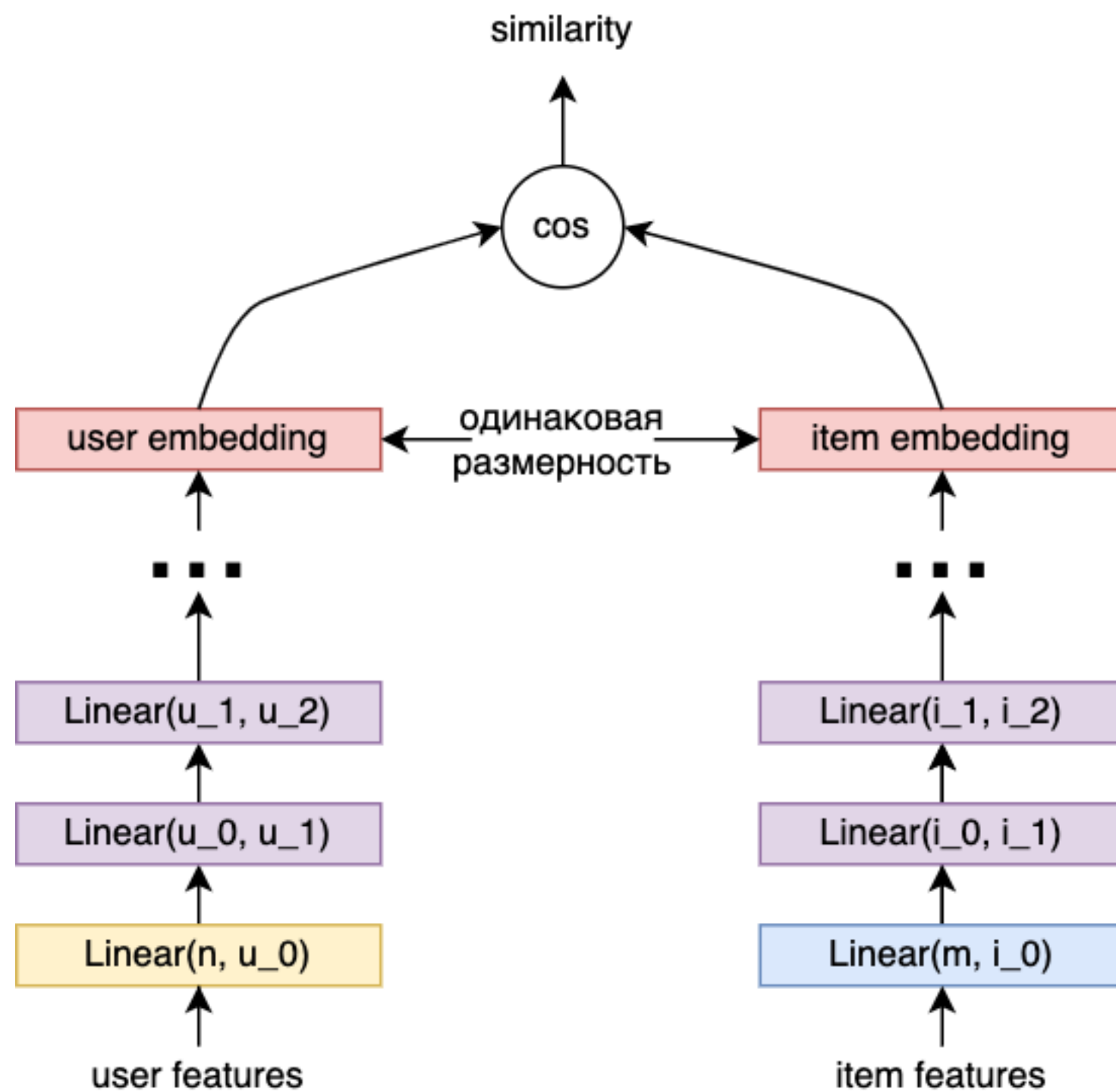


user $item_0^-$ $item_1^-$ $item_2^-$ $item^+$ $item_3^-$

 ↓

 -0.0218

Deep Structured Semantic Model

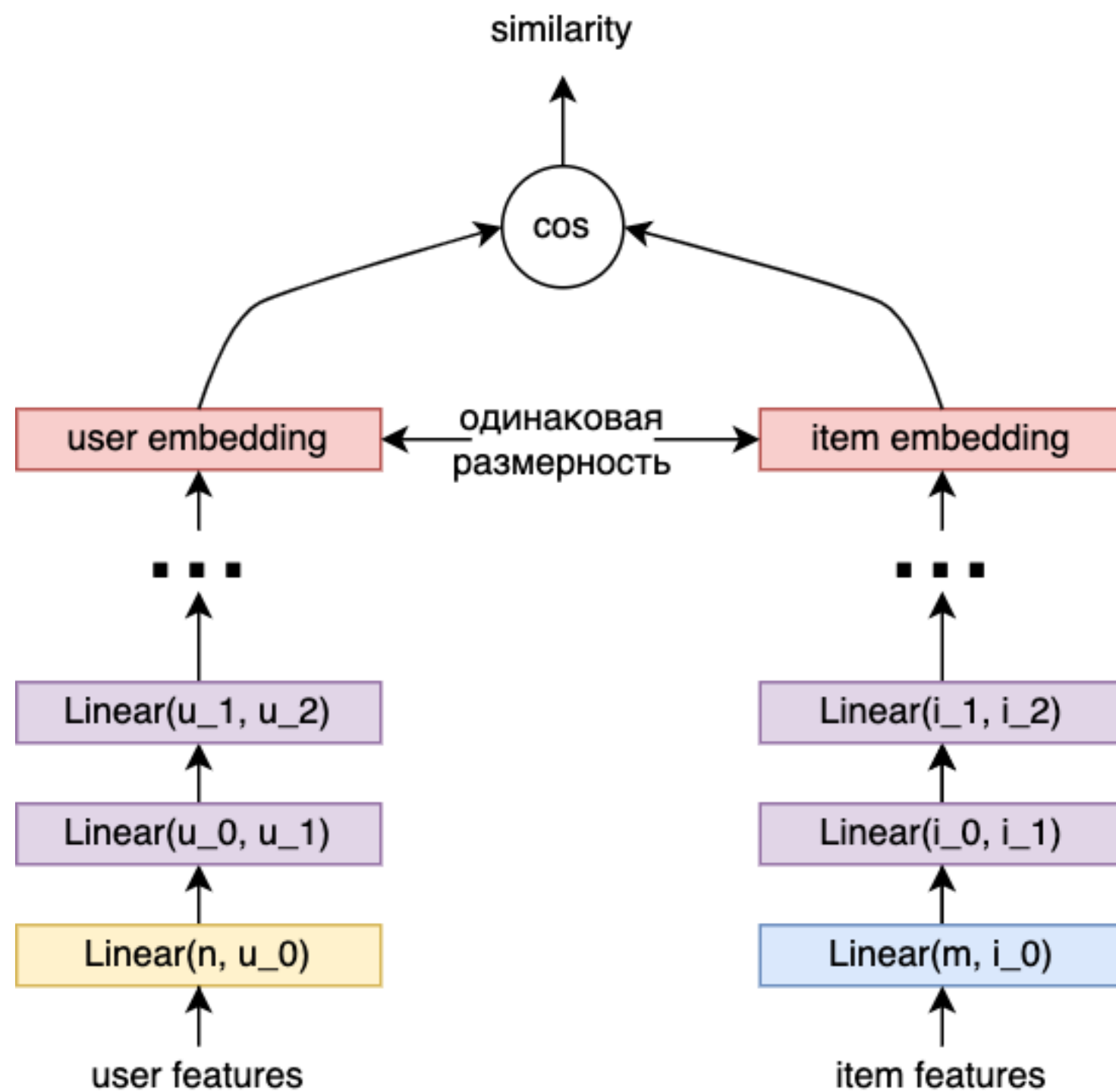


user $item_0^-$ $item_1^-$ $item_2^-$ $item^+$ $item_3^-$

-0.0218 -0.4689

A yellow arrow points from the $item_1^-$ label to the value -0.4689.

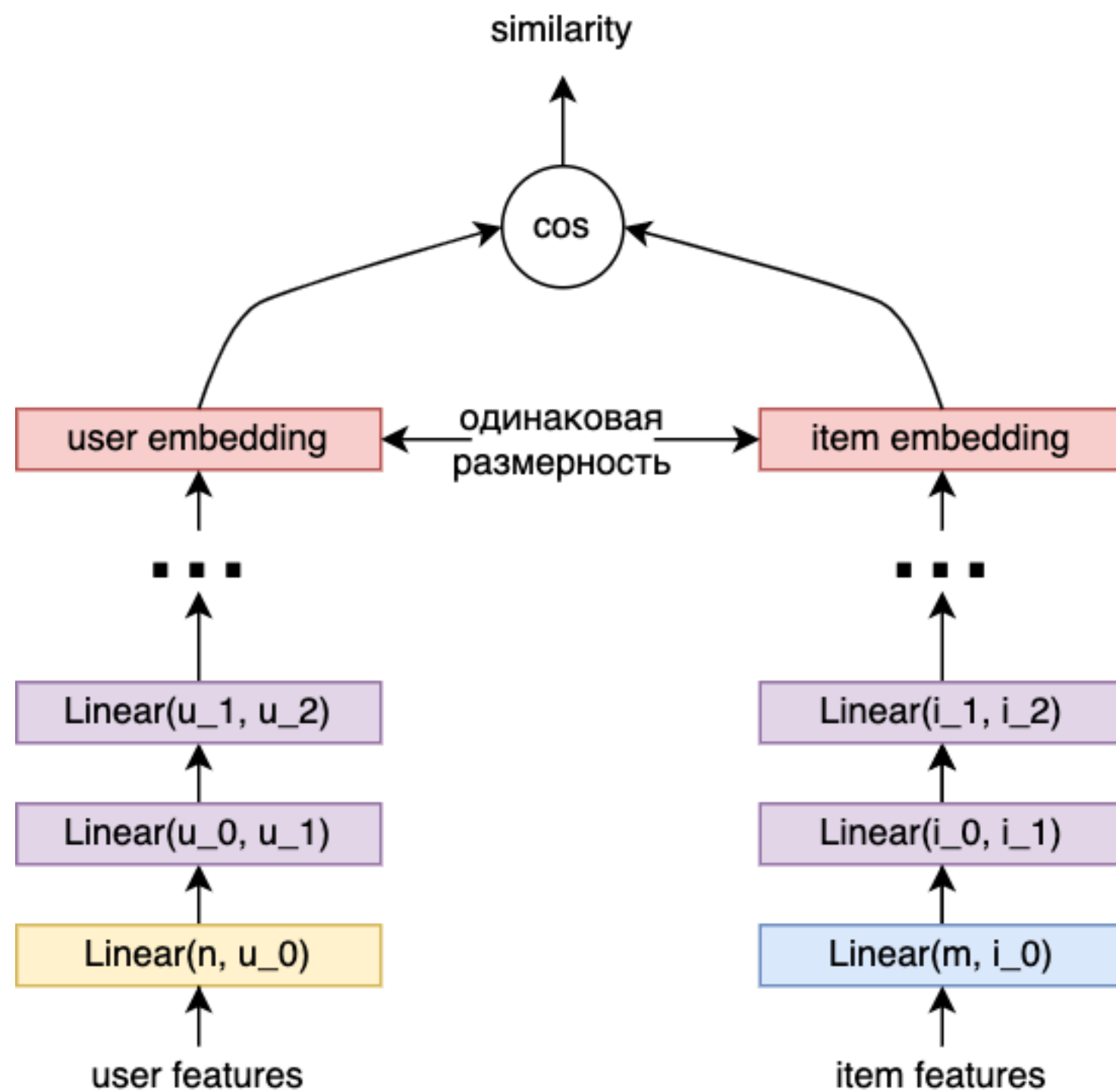
Deep Structured Semantic Model



user $item_0^-$ $item_1^-$ $item_2^-$ $item^+$ $item_3^-$

-0.0218 -0.4689 0.4136

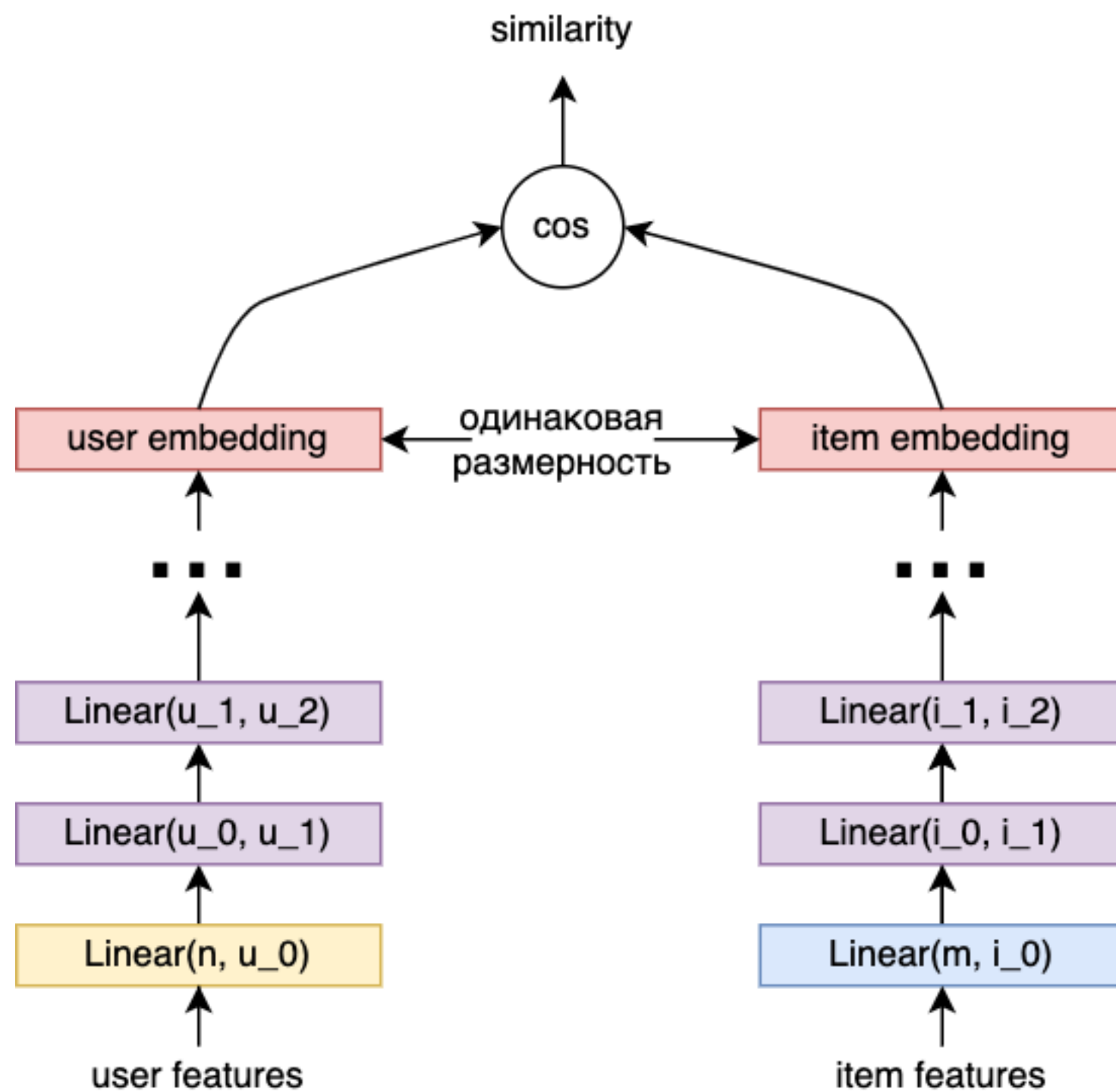
Deep Structured Semantic Model



user $item_0^-$ $item_1^-$ $item_2^-$ $item^+$ $item_3^-$

-0.0218 -0.4689 0.4136 0.7126

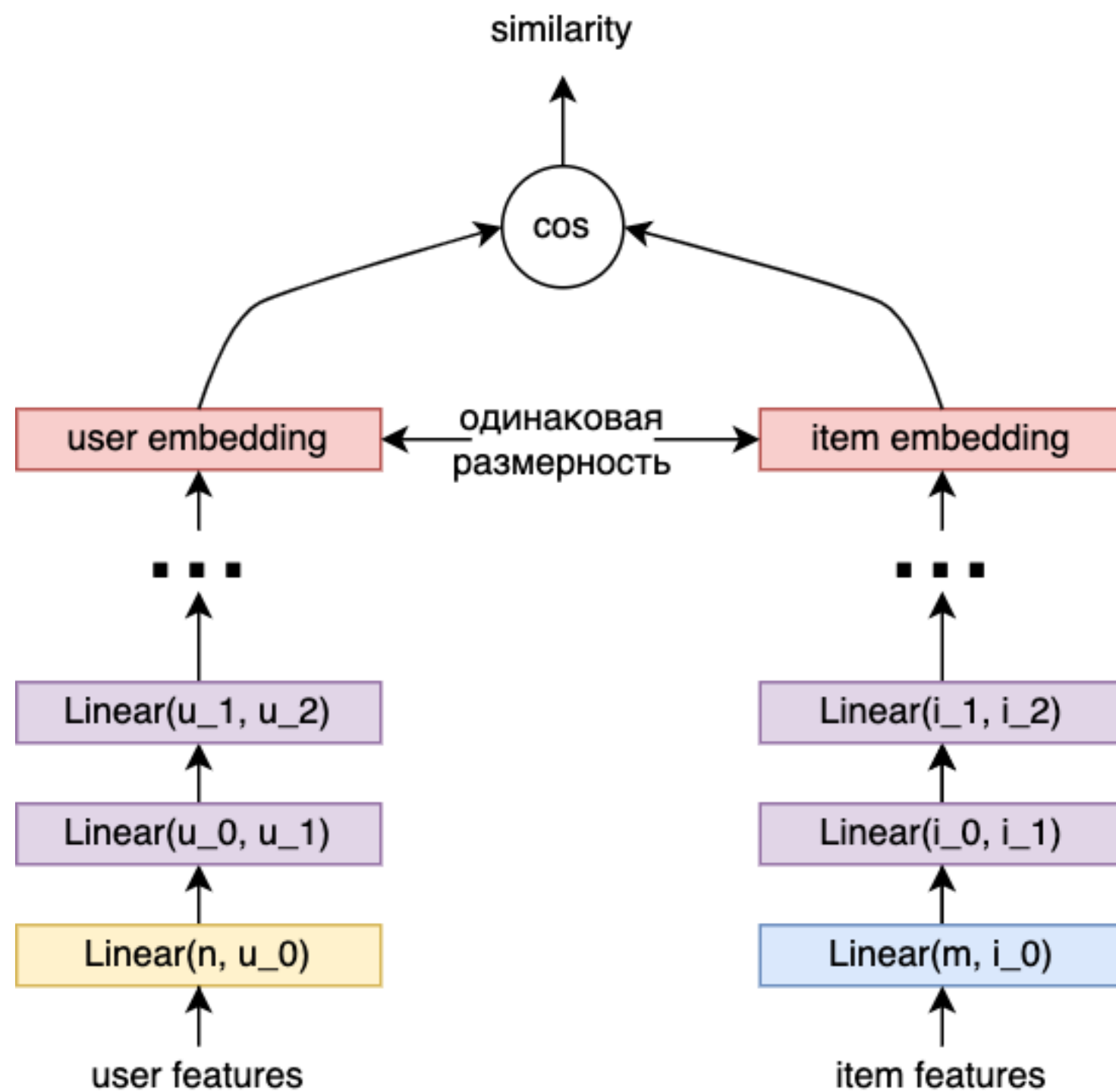
Deep Structured Semantic Model



user	$item_0^-$	$item_1^-$	$item_2^-$	$item^+$	$item_3^-$
	-0.0218	-0.4689	0.4136	0.7126	-0.6863

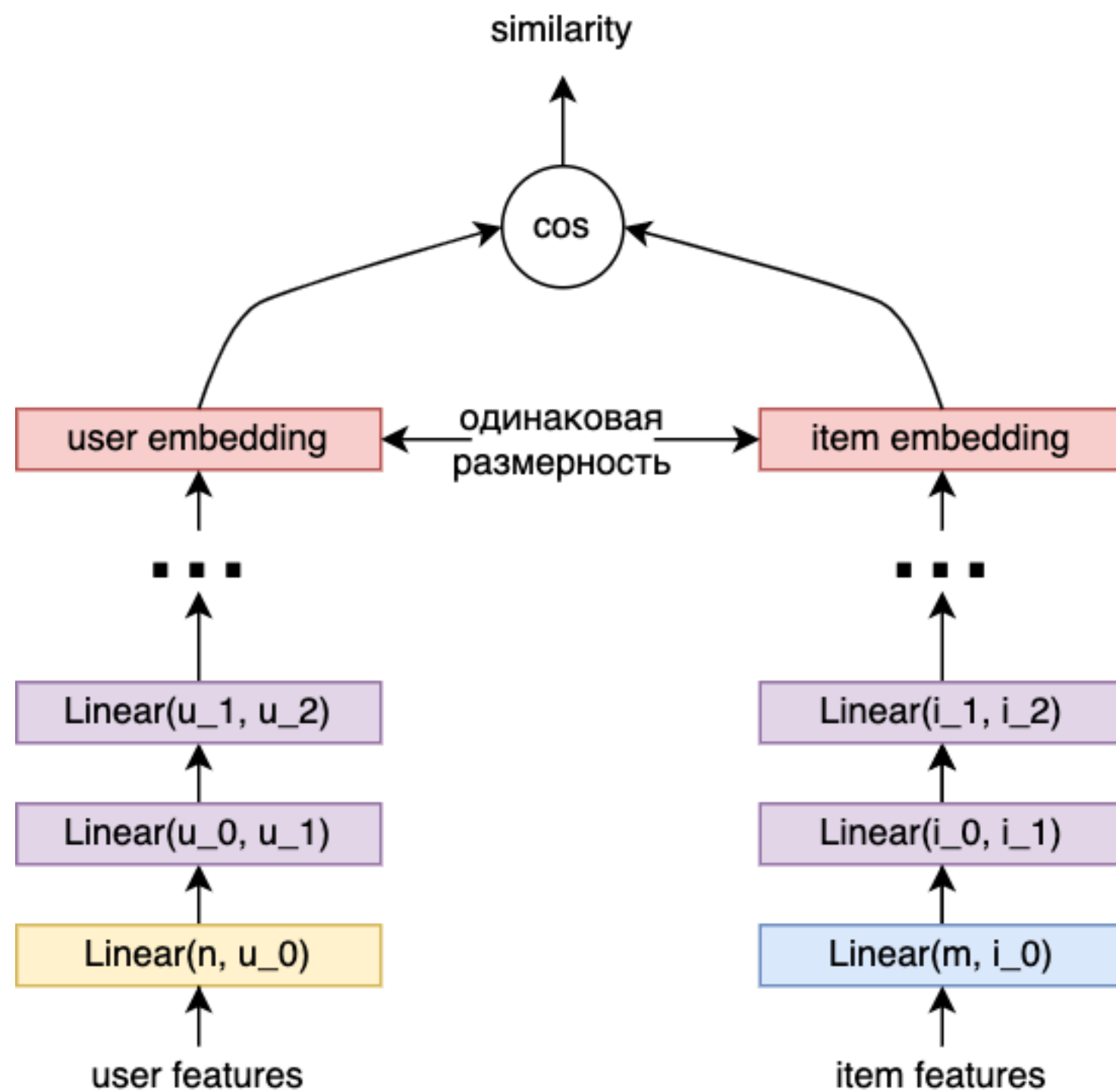
A yellow arrow points from the 'user' column to the $item_3^-$ value (-0.6863).

Deep Structured Semantic Model



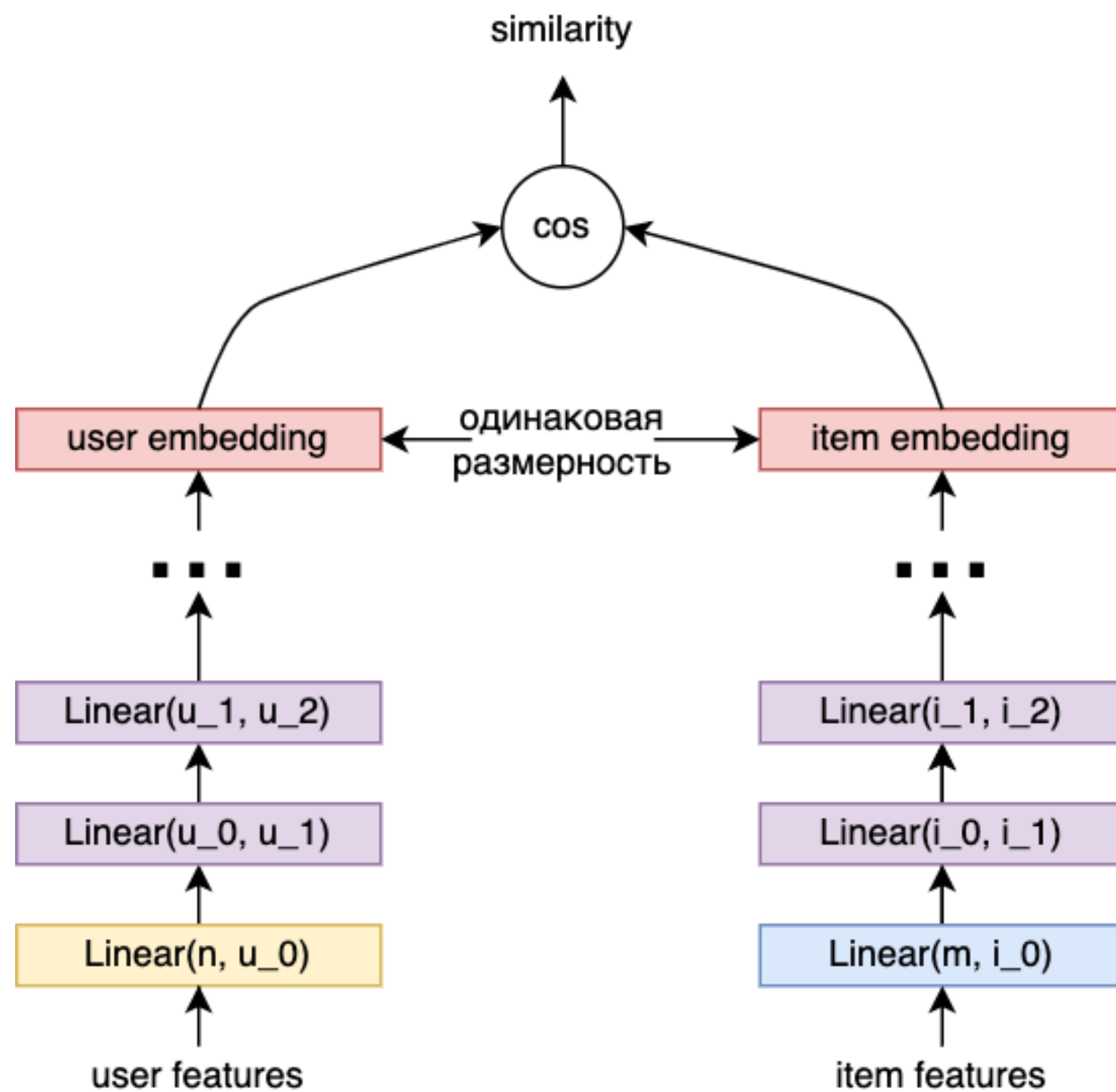
user	$item_0^-$	$item_1^-$	$item_2^-$	$item^+$	$item_3^-$
	-0.0218	-0.4689	0.4136	0.7126	-0.6863
$Softmax(\gamma * x)$					
	0.1729	0.1106	0.2672	0.3604	
	0.0890				

Deep Structured Semantic Model



user	$item_0^-$	$item_1^-$	$item_2^-$	$item^+$	$item_3^-$
	-0.0218	-0.4689	0.4136	0.7126	-0.6863
	$Softmax(\gamma * x)$				
	0.1729	0.1106	0.2672	0.3604	
	0.0890				

Deep Structured Semantic Model



user $item_0^-$ $item_1^-$ $item_2^-$ $item^+$ $item_3^-$

-0.0218 -0.4689 0.4136 0.7126 -0.6863

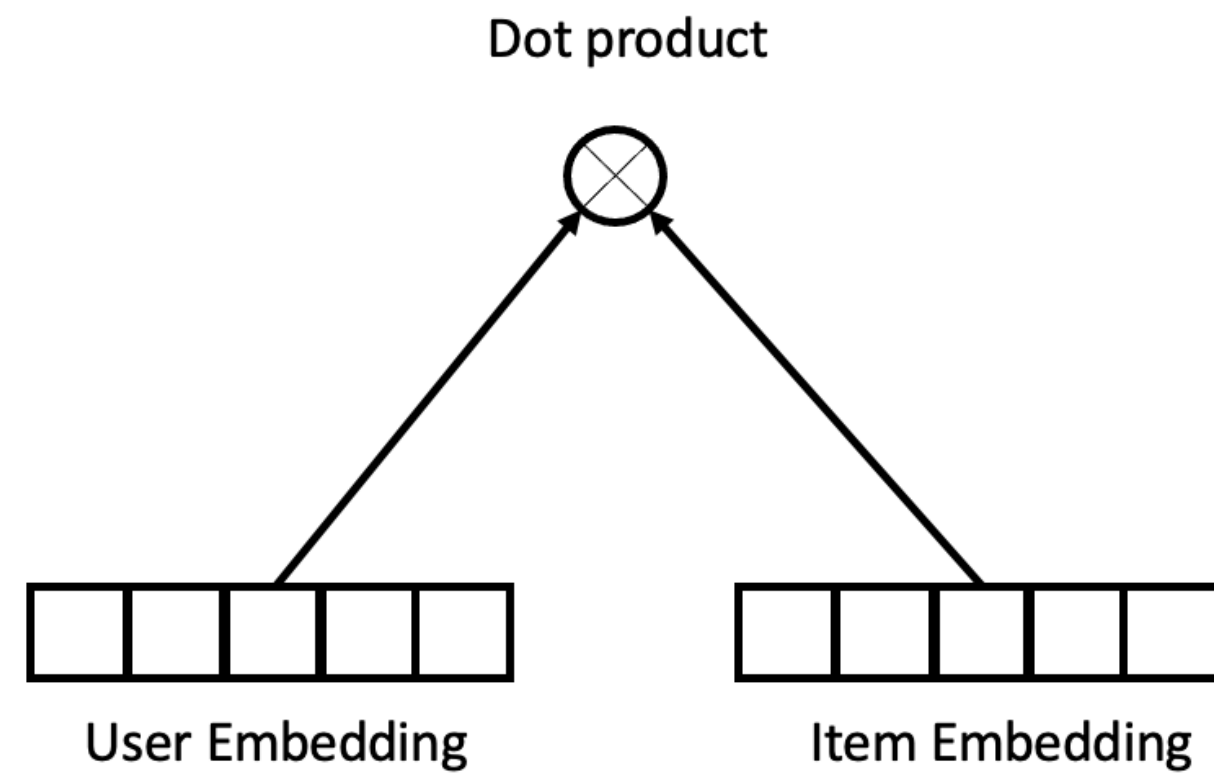
$\text{Softmax}(\gamma * x)$

0.1729 0.1106 0.2672 0.3604
0.0890

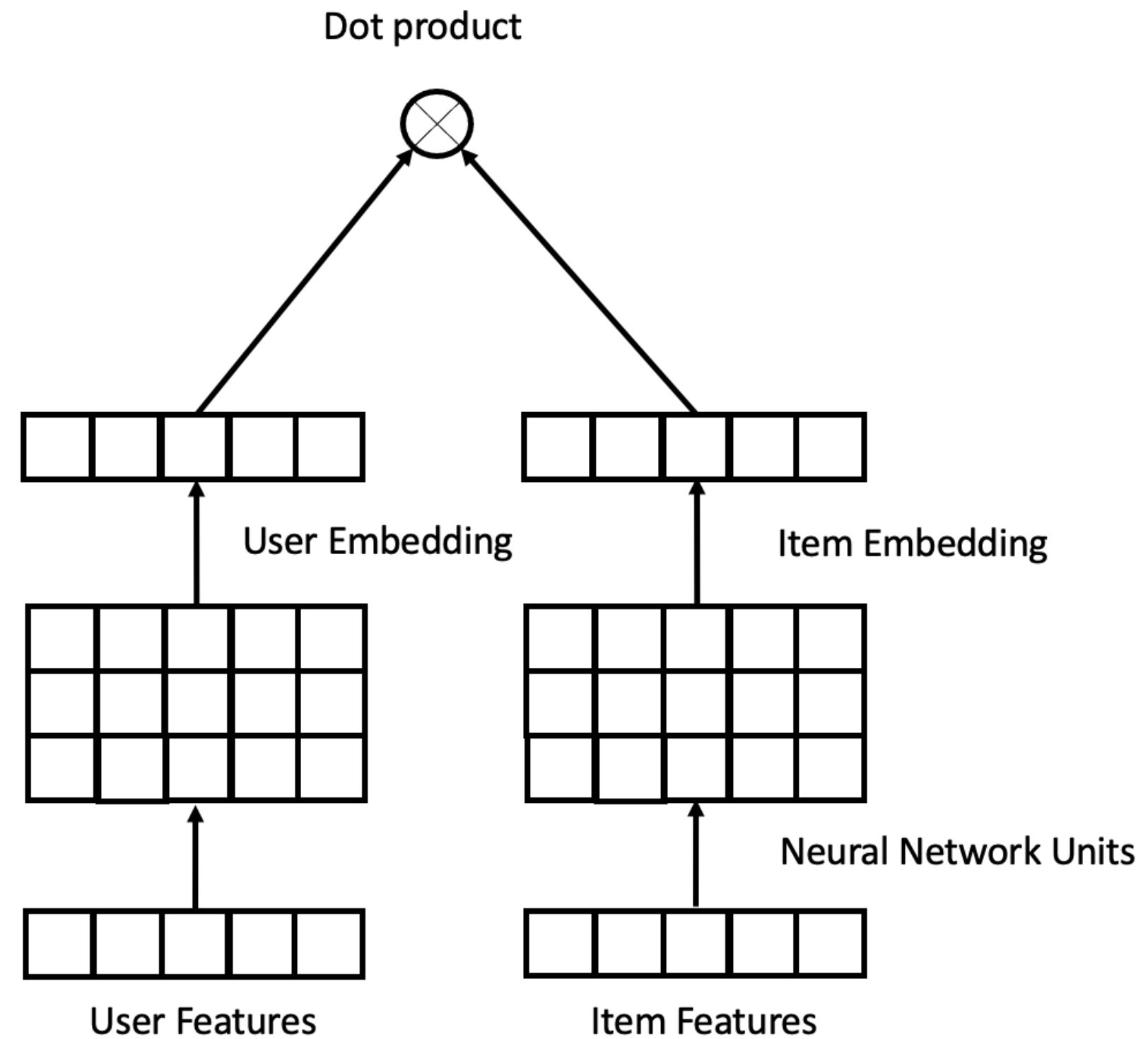
$$L = -\log(0.3604) = 1.02$$

Deep Structured Semantic Model

a) Matrix Factorization task



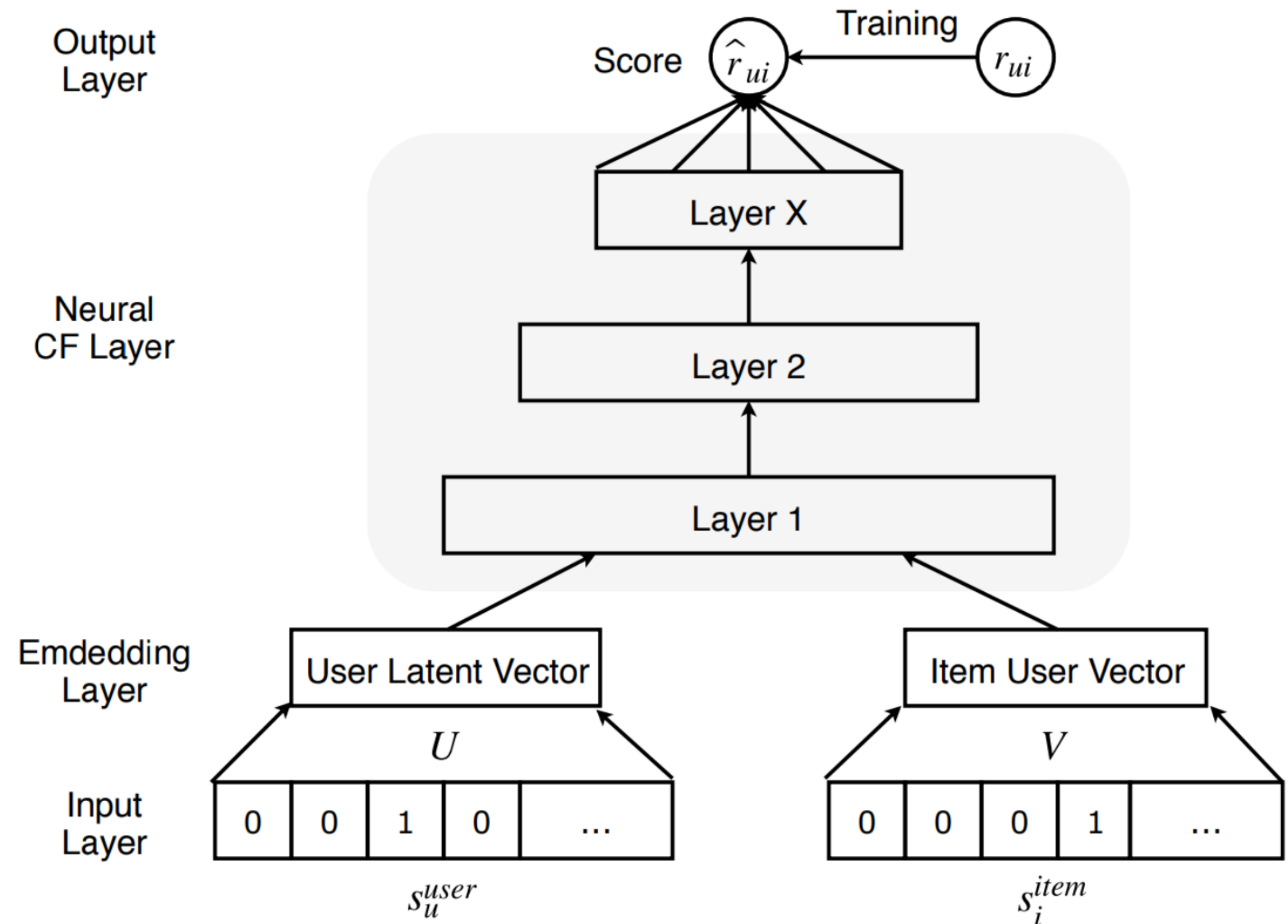
b) DSSM task



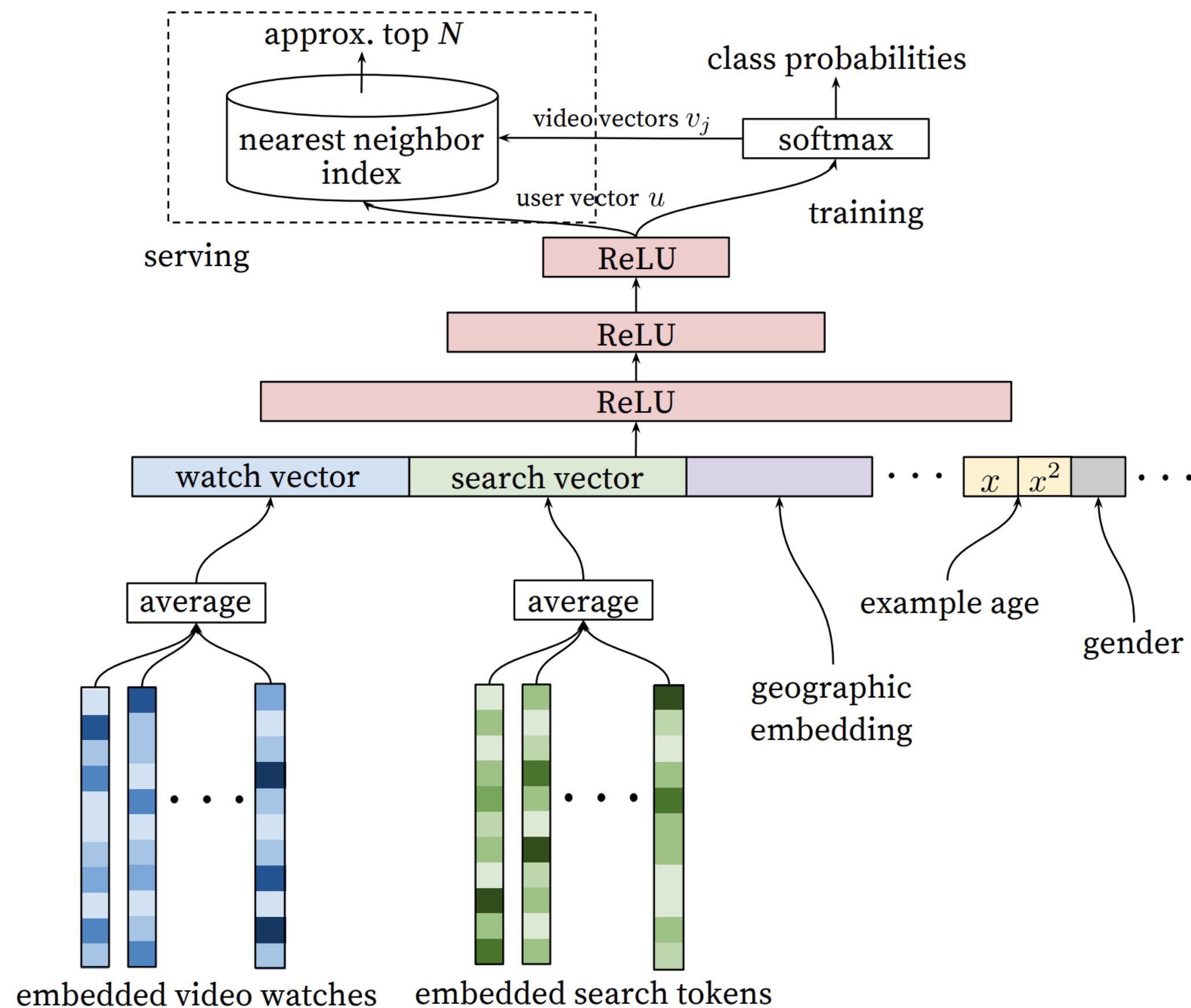
NOT Deep Structured Semantic Model



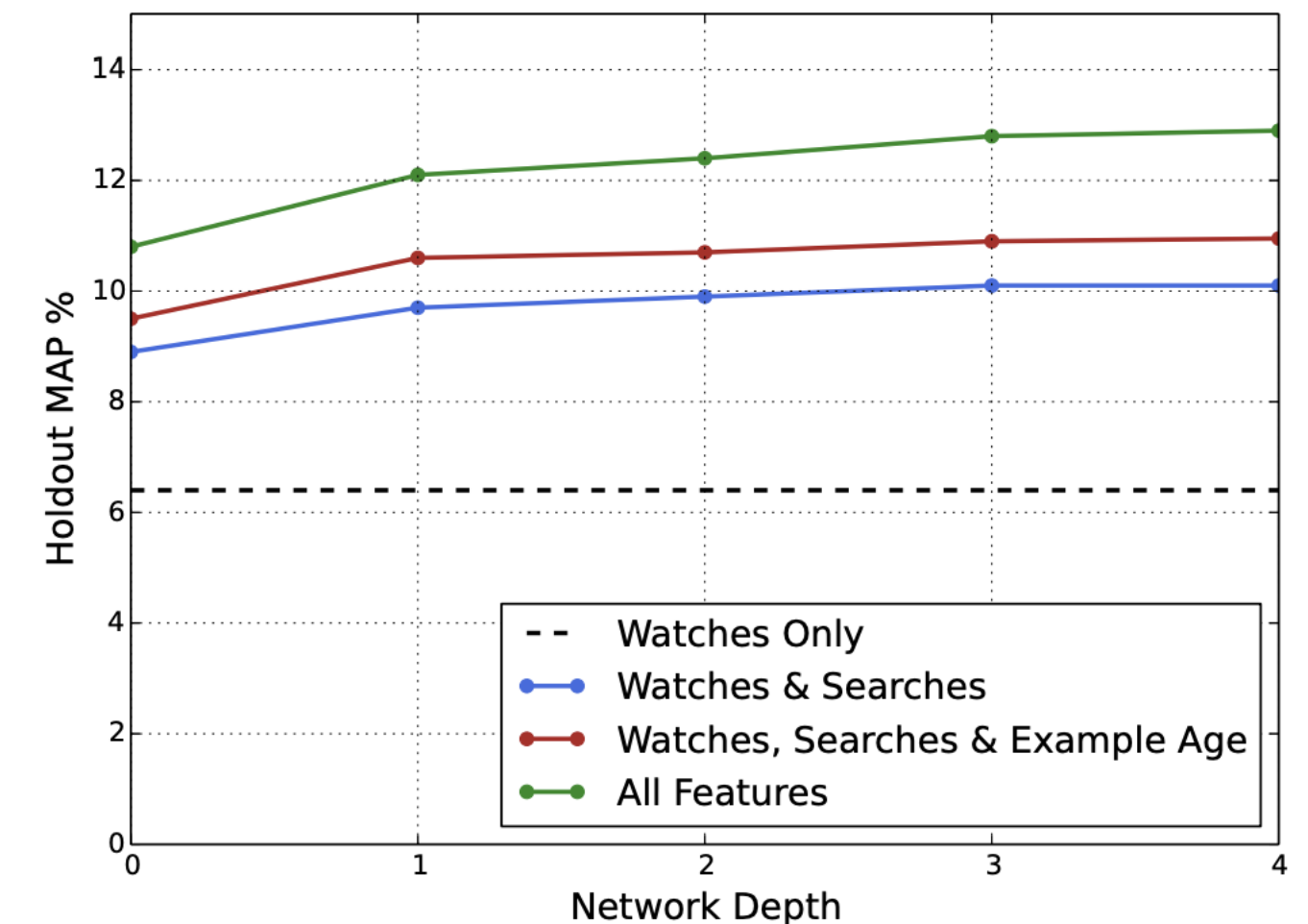
ЭТО НЕ DSSM!
ЭТО NCF



Deep Neural Networks for YouTube Recommendations



- Depth 0: A linear layer simply transforms the concatenation layer to match the softmax dimension of 256
- Depth 1: 256 ReLU
- Depth 2: 512 ReLU → 256 ReLU
- Depth 3: 1024 ReLU → 512 ReLU → 256 ReLU
- Depth 4: 2048 ReLU → 1024 ReLU → 512 ReLU → 256 ReLU



$$P(w_t = i | U, C) = \frac{e^{v_i u}}{\sum_{j \in V} e^{v_j u}}$$

Deep Neural Networks for YouTube Recommendations

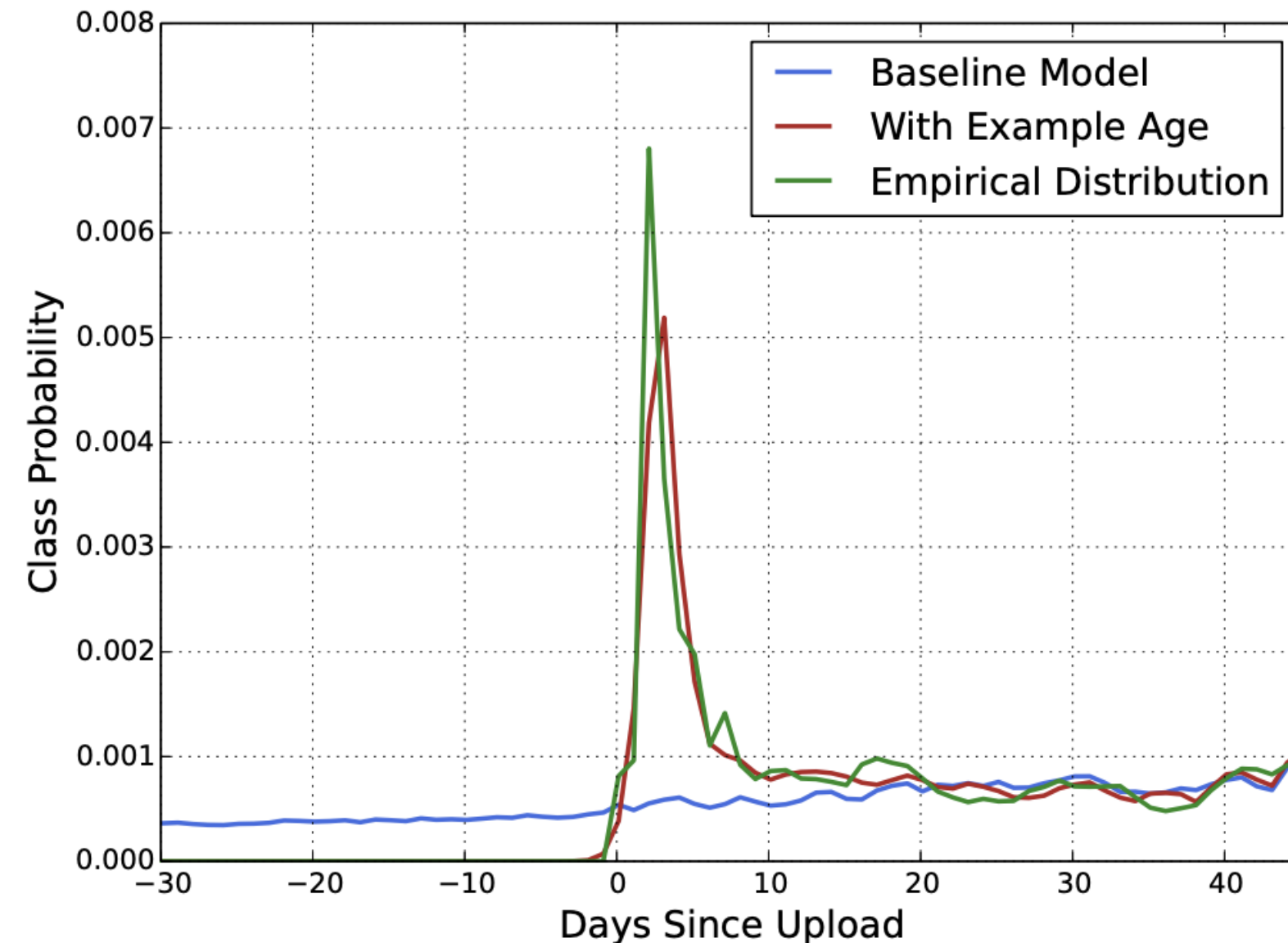


Figure 4: For a given video [26], the model trained with example age as a feature is able to accurately represent the upload time and time-dependant popularity observed in the data. Without the feature, the model would predict approximately the average likelihood over the training window.

Hidden layers	weighted, per-user loss
None	41.6%
256 ReLU	36.9%
512 ReLU	36.7%
1024 ReLU	35.8%
512 ReLU $\rightarrow$ 256 ReLU	35.2%
1024 ReLU $\rightarrow$ 512 ReLU	34.7%
1024 ReLU $\rightarrow$ 512 ReLU $\rightarrow$ 256 ReLU	34.6%

Table 1: Effects of wider and deeper hidden ReLU layers on watch time-weighted pairwise loss computed on next-day holdout data.

Deep Neural Networks for YouTube Recommendations

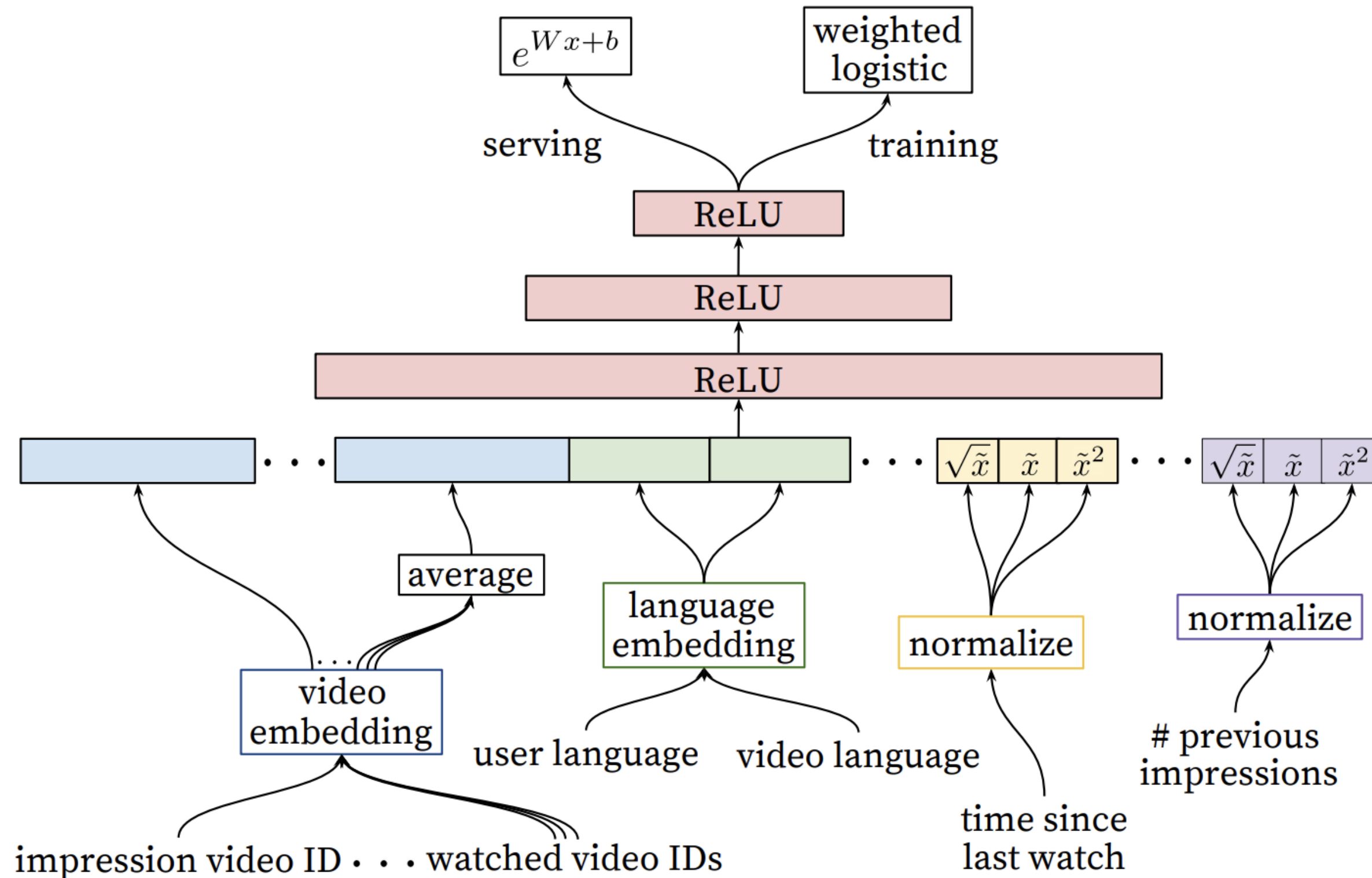


Figure 7: Deep ranking network architecture depicting embedded categorical features (both univalent and multivalent) with shared embeddings and powers of normalized continuous features. All layers are fully connected. In practice, hundreds of features are fed into the network.

Как добавить контетн/контекст в коллаборативную фильрацию?



Как добавить контетн/контекст в коллаборативную фильрацию?



User_id	Item_id	Date
5	4	2024-02-24
6	1	2024-02-24
5	3	2024-02-22
1	2	2024-02-13
5	1	2024-02-10
4	2	2024-02-08
2	3	2024-02-08
2	1	2024-02-07
....		
1	4	2023-12-31



U/I	item 1	item2	item 3	item 4
User 1	0	1	0	1
User 2	1	0	1	0
User 3	0	1	0	1
User 4	0	1	0	0
User 5	1	0	1	1
User 6	1	1	0	1
User 7	0	1	1	0
User 8	1	0	0	1

Как добавить контетн/контекст в коллаборативную фильрацию?

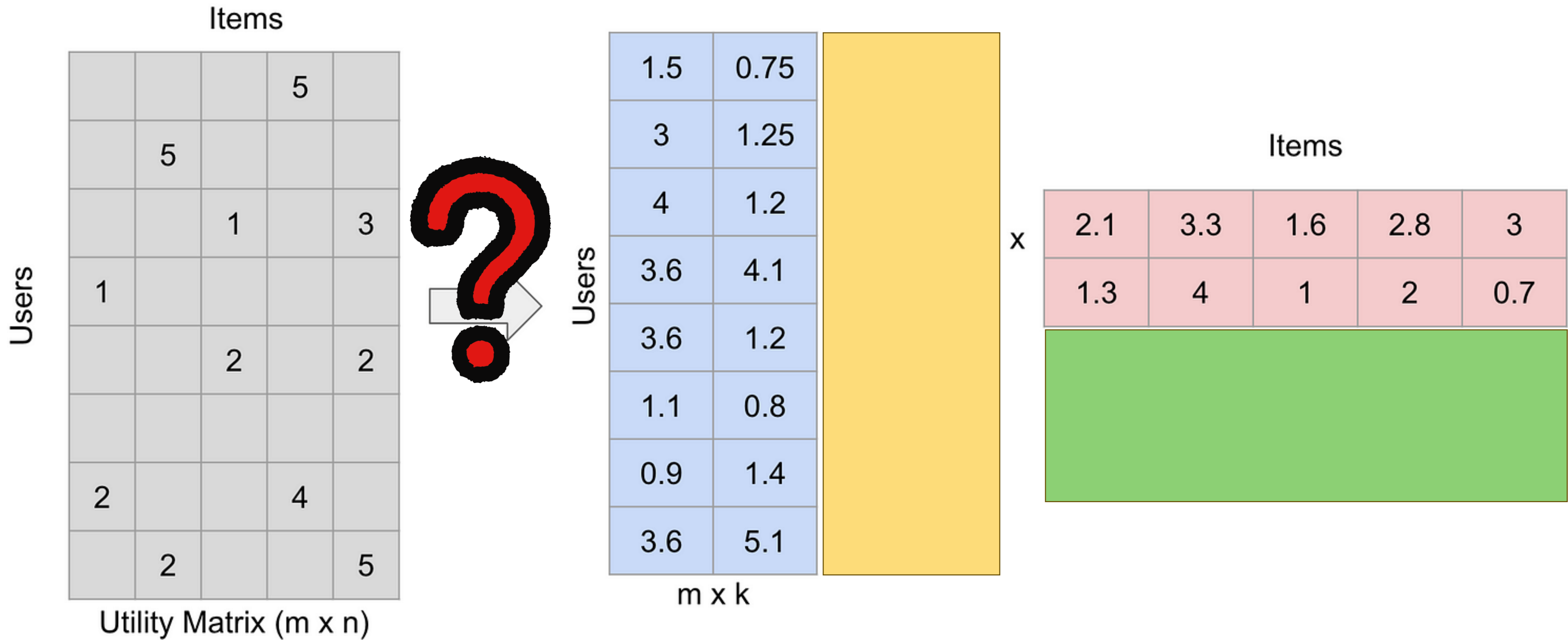


User_id	Item_id	Date
5	4	2024-02-24
6	1	2024-02-24
5	3	2024-02-22
1	2	2024-02-13
5	1	2024-02-10
4	2	2024-02-08
2	3	2024-02-08
2	1	2024-02-07
....		
1	4	2023-12-31

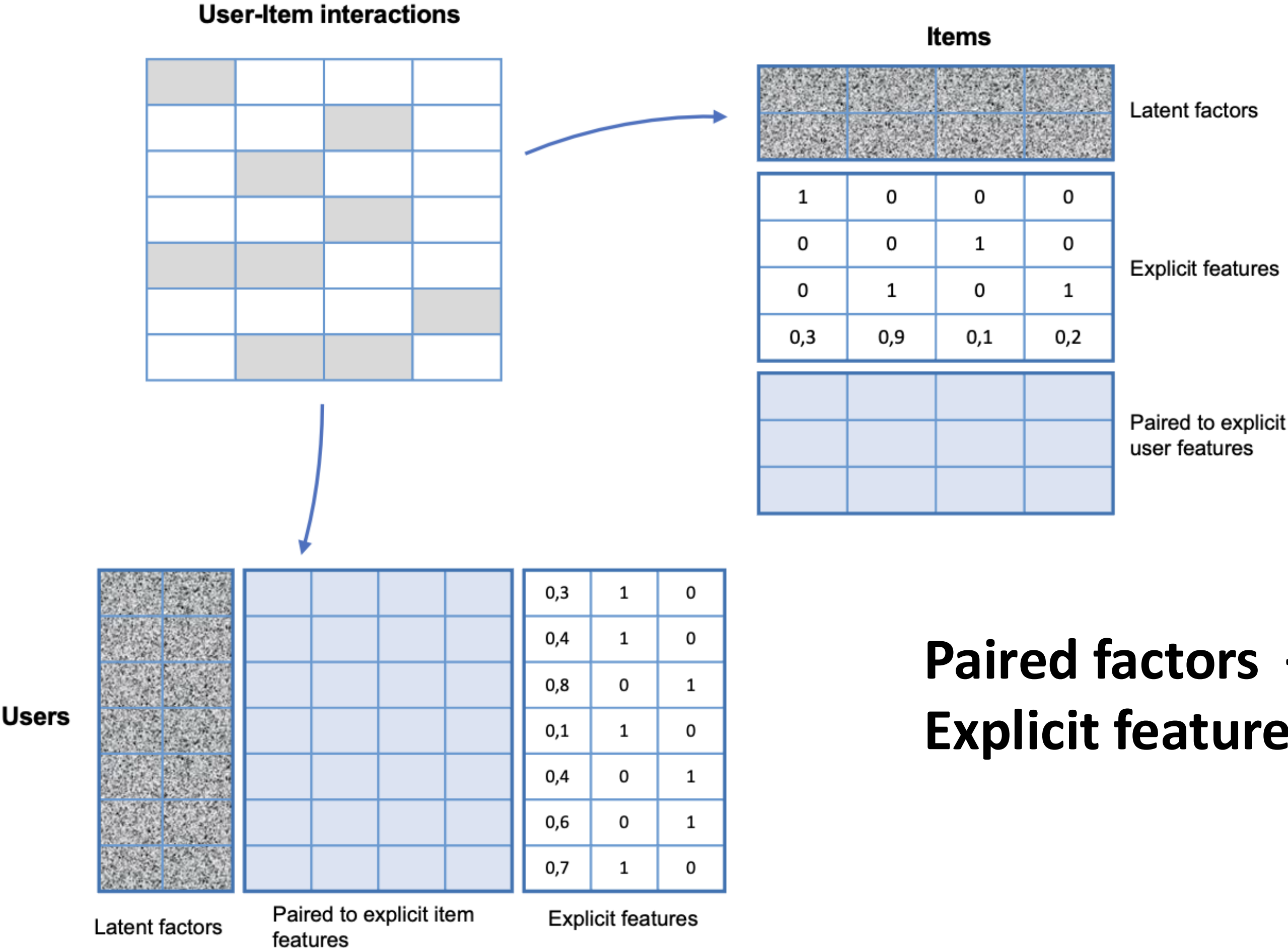


U/I	item 1	item2	item 3	item 4
User 1	0	1	0	0.5
User 2	0.5	0	1	0
User 3	0	1	0	1
User 4	0	1	0	0
User 5	0.33	0	0.5	1
User 6	1	0.33	0	0.5
User 7	0	0.5	1	0
User 8	0.5	0	0	1

Как добавить контетн/контекст в коллаборативную фильрацию?

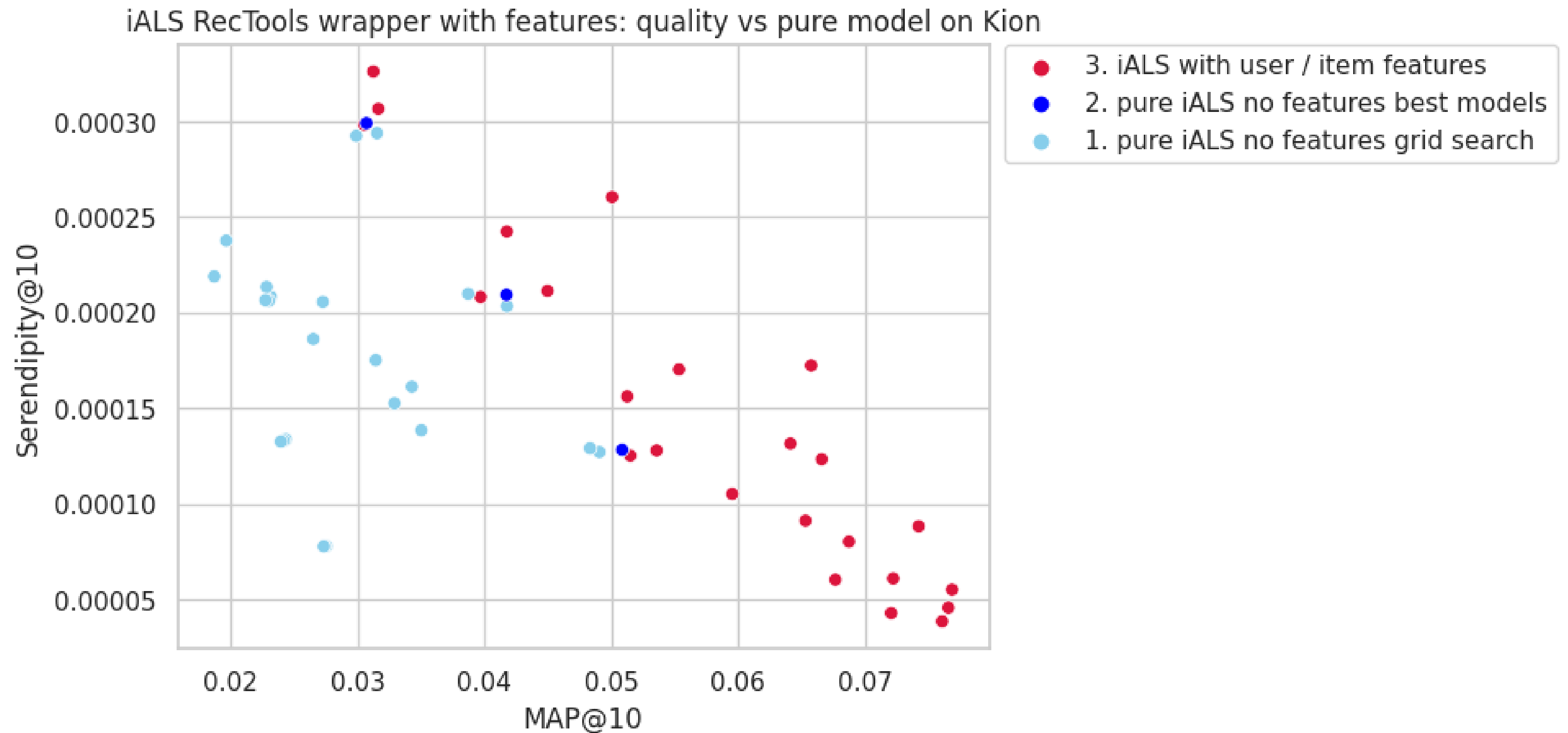


Hybrid iALS



Paired factors – обучаемые.
Explicit features – заморожены.

Content iALS



Metadata Embeddings for User and Item Cold-start Recommendations (LightFM)

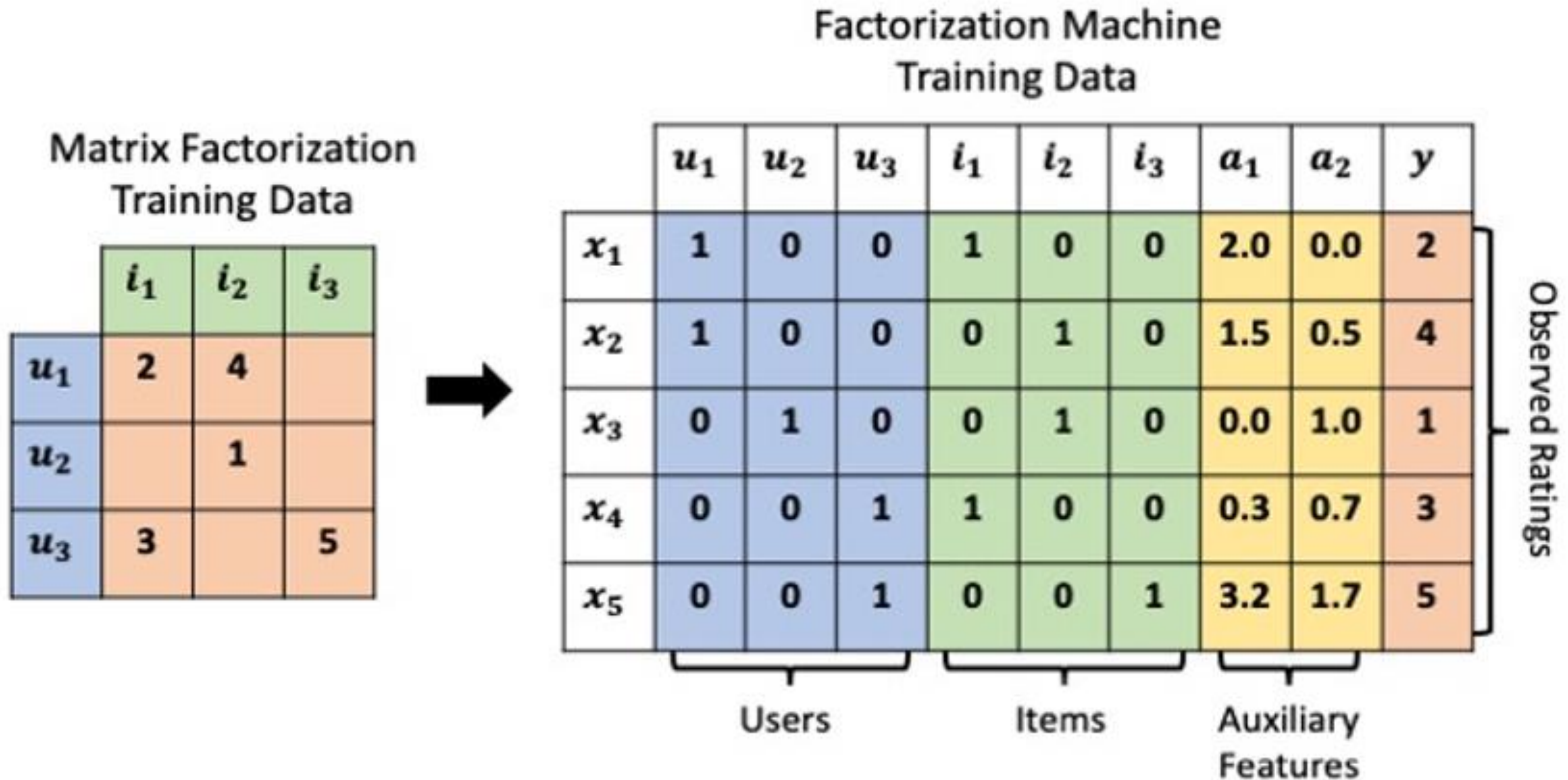


2.1 Motivation

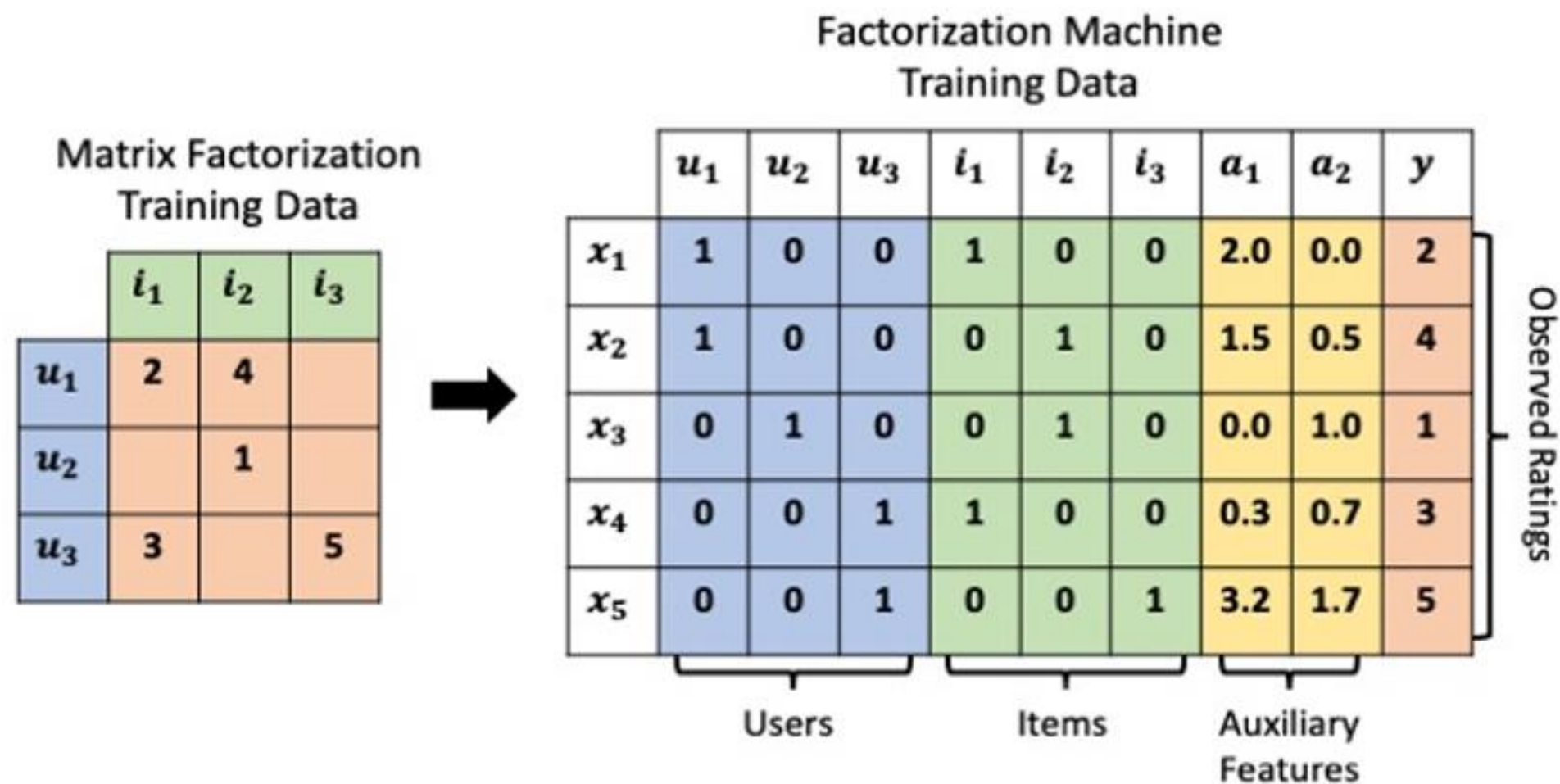
The structure of the LightFM model is motivated by two considerations.

1. The model must be able to learn user and item representations from interaction data: if items described as ‘ball gown’ and ‘pencil skirt’ are consistently all liked by users, the model must learn that ball gowns are similar to pencil skirts.
2. The model must be able to compute recommendations for new items and users.

Factorization Machines



Factorization Machines



x_p – p -я компонента вектора x_i

v_p – эмбеddинг размерности k

$$f(x) = w_0 + \sum_{p=1}^P w_p x_p + \sum_{p=1}^{P-1} \sum_{q=p+1}^P \langle v_p, v_q \rangle x_p x_q$$

Factorization Machines



Feature vector $\mathbf{x}$															Target y							
$\mathbf{x}^{(1)}$	1	0	0	...	1	0	0	0	...	0.3	0.3	0.3	0	...	13	0	0	0	0	...	5	$y^{(1)}$
$\mathbf{x}^{(2)}$	1	0	0	...	0	1	0	0	...	0.3	0.3	0.3	0	...	14	1	0	0	0	...	3	$y^{(2)}$
$\mathbf{x}^{(3)}$	1	0	0	...	0	0	1	0	...	0.3	0.3	0.3	0	...	16	0	1	0	0	...	1	$y^{(2)}$
$\mathbf{x}^{(4)}$	0	1	0	...	0	0	1	0	...	0	0	0.5	0.5	...	5	0	0	0	0	...	4	$y^{(3)}$
$\mathbf{x}^{(5)}$	0	1	0	...	0	0	0	1	...	0	0	0.5	0.5	...	8	0	0	1	0	...	5	$y^{(4)}$
$\mathbf{x}^{(6)}$	0	0	1	...	1	0	0	0	...	0.5	0	0.5	0	...	9	0	0	0	0	...	1	$y^{(5)}$
$\mathbf{x}^{(7)}$	0	0	1	...	0	0	1	0	...	0.5	0	0.5	0	...	12	1	0	0	0	...	5	$y^{(6)}$
	A	B	C	...	TI	NH	SW	ST	...	TI	NH	SW	ST	...		TI	NH	SW	ST	...		
	User				Movie					Other Movies rated					Time	Last Movie rated						

Metadata Embeddings for User and Item Cold-start Recommendations (LightFM)



Формализация.

U - множество пользователей,

I - множество объектов,

F^U - множество признаков пользователей,

F^I - множество признаков объектов.

Все пары $(u, i) \in U \times I$ - это объединение всех положительных S^+ и отрицательных S^- взаимодействий.

Каждый пользователь описан набором заранее известных признаков (мета данных) $f_u \subset F^U$, то же самое для объектов $f_i \subset F^I$.

Metadata Embeddings for User and Item Cold-start Recommendations (LightFM)



Для каждого признака f мы задаем вектор размерности d отдельно для пользователей и отдельно для айтемов (e_f^U и e_f^I соответственно).

Латентное представление пользователя представлено суммой его латентных векторов признаков:

$$q_u = \sum_{j \in f_u} e_j^U$$

Аналогично для объектов:

$$p_i = \sum_{j \in f_i} e_j^I$$

Так же, по пользователю и объекту есть смещения (bias):

$$b_u = \sum_{j \in f_u} b_j^U$$

$$b_i = \sum_{j \in f_i} b_j^I$$

Предсказание из модели будет получать через скалярное произведение эмбедингов пользователя и объекта.

$$\hat{r}_{ui} = f(q_u \cdot p_i + b_u + b_i)$$

Metadata Embeddings for User and Item Cold-start Recommendations (LightFM)



Функция $f()$ может быть разной, автор статьи выбрал сигмоиду, поскольку использовал бинарные данные.

$$f(x) = \frac{1}{1 + \exp(-x)}$$

Задача оптимизации будет сформулирована как максимизация правдоподобия (данных при параметрах), с обучением модели с помощью стохастического градиентного спуска.

$$L(e^U, e^I, b^U, b^I) = \prod_{(u,i) \in S^+} \hat{r}_{ui} \cdot \prod_{(u,i) \in S^-} (1 - \hat{r}_{ui})$$

Стоит отметить, что **оригинальная версия LightFM поддерживает только категориальные признаки** (из-за механики эмбедингов)

Metadata Embeddings for User and Item Cold-start Recommendations (LightFM)



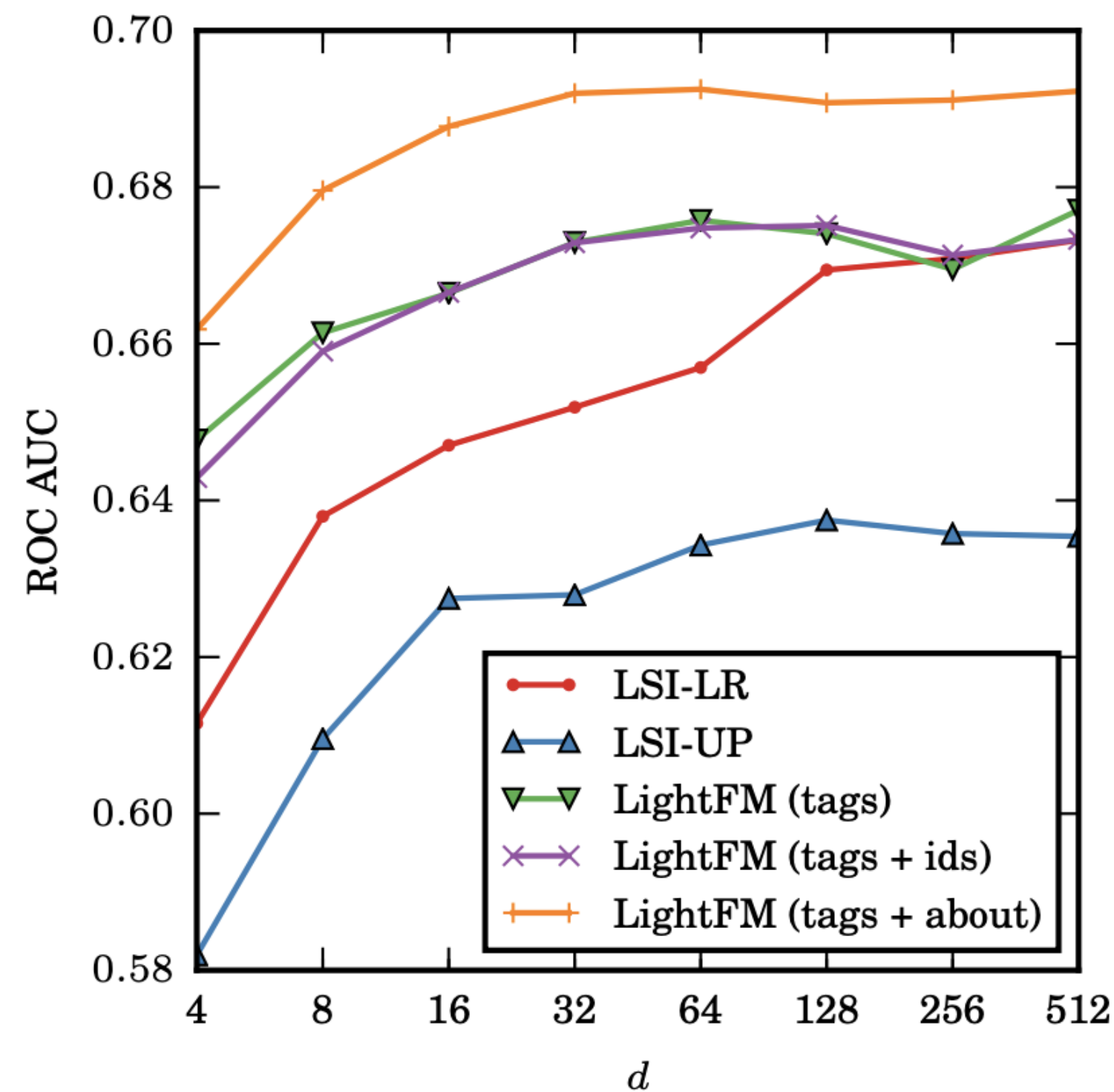
4. **LightFM (tags)**: the LightFM model using only tag features.
5. **LightFM (tags + ids)**: the LightFM model using both tag and item indicator features.
6. **LightFM (tags + about)**: the LightFM model using both item and user features. User features are available only for the CrossValidated dataset. I construct them by converting the 'About Me' sections of users' profiles to a bag-of-words representation. I first strip them of all HTML tags and non-alphabetical characters, then convert the resulting string to lowercase and tokenise on spaces.

Metadata Embeddings for User and Item Cold-start Recommendations (LightFM)

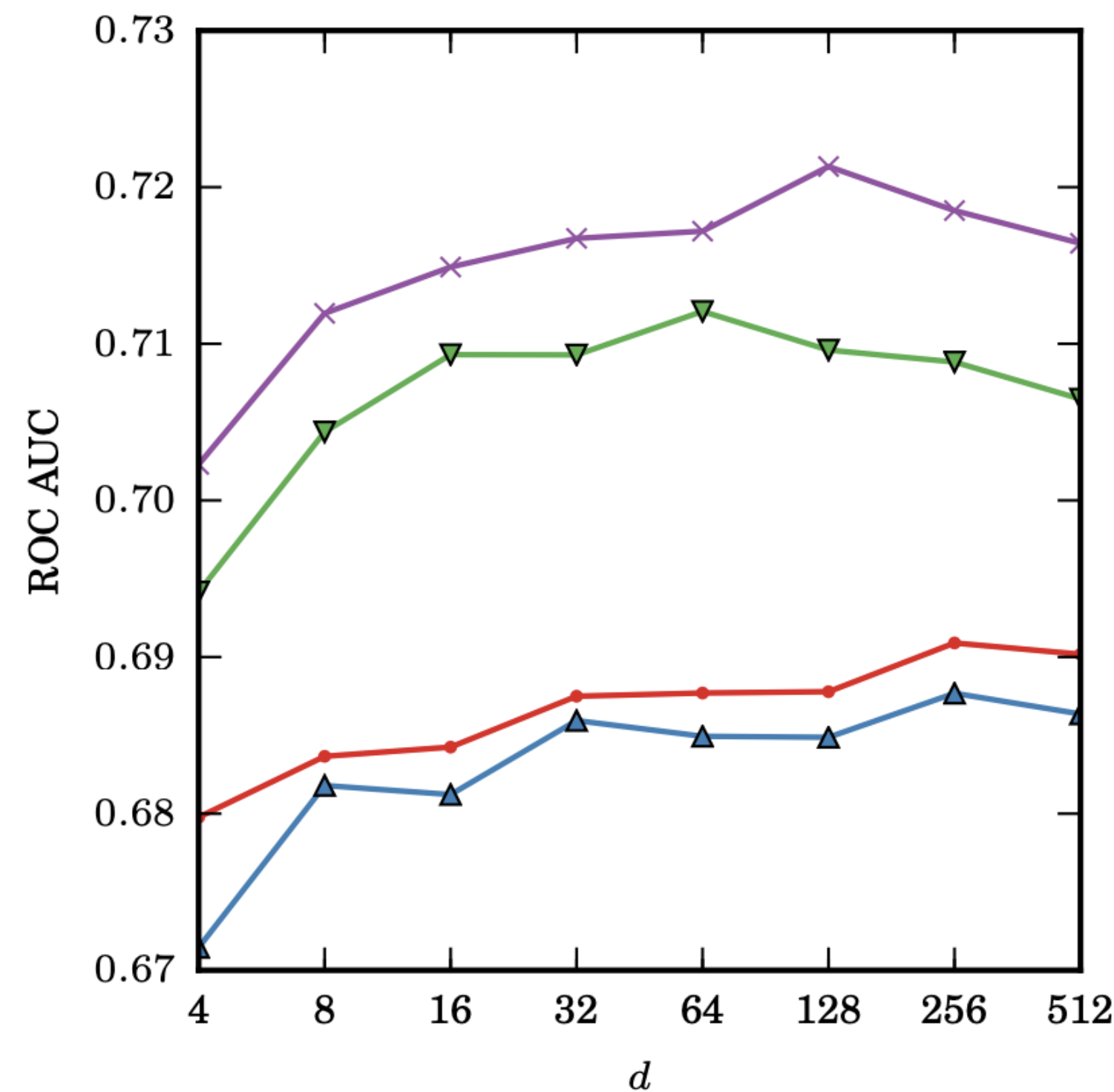


	CrossValidated		MovieLens	
	Warm	Cold	Warm	Cold
LSI-LR	0.662	0.660	0.686	0.690
LSI-UP	0.636	0.637	0.687	0.681
MF	0.541	0.508	0.762	0.500
LightFM (tags)	0.675	0.675	0.744	0.707
LightFM (tags + ids)	0.682	0.674	0.763	0.716
LightFM (tags + about)	0.695	0.696		

Metadata Embeddings for User and Item Cold-start Recommendations (LightFM)



(a) CrossValidated



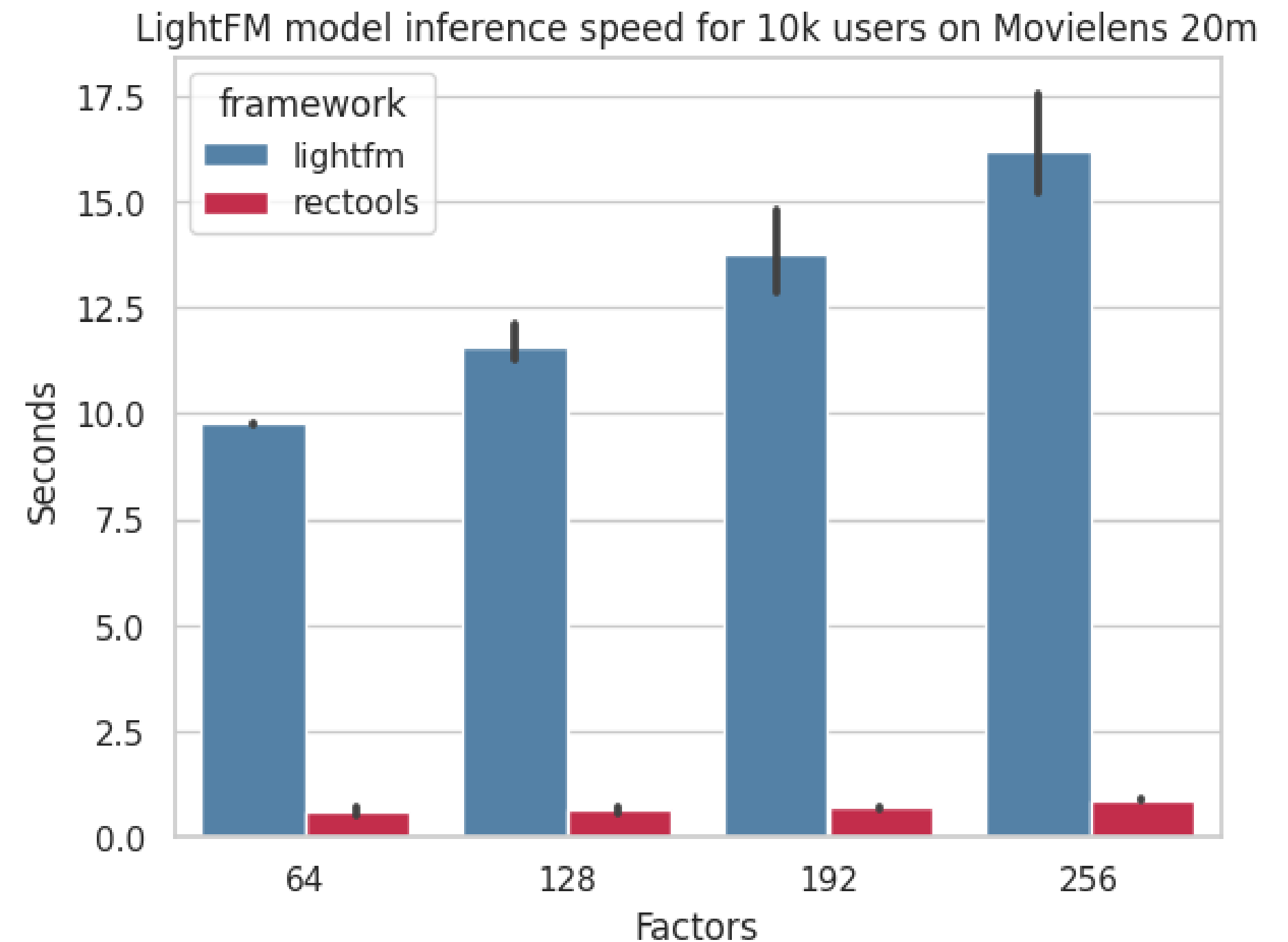
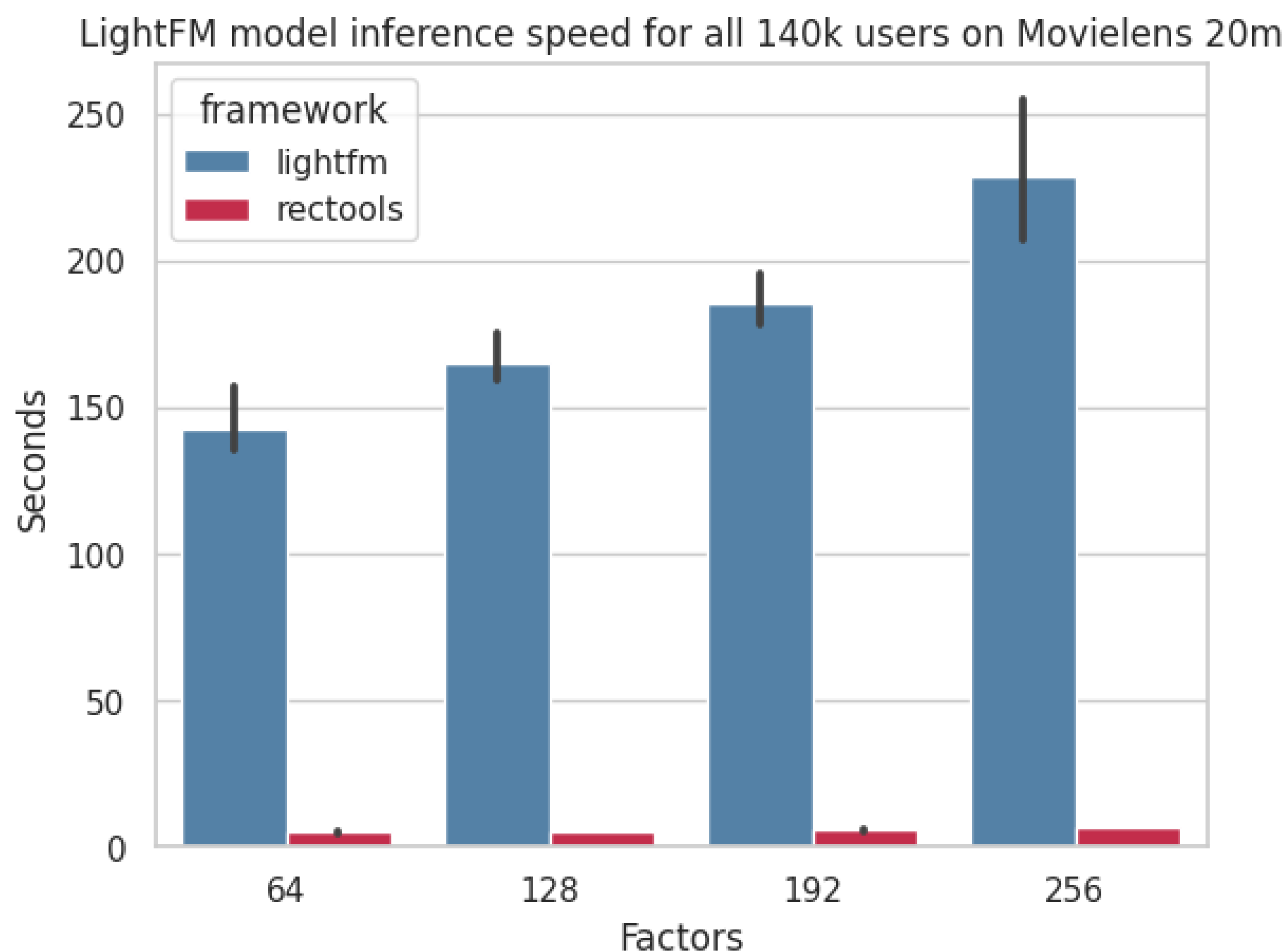
(b) MovieLens

Реализации LightFM



Оригинальная от автора статьи: <https://making.lyst.com/lightfm/docs/home.html>

Современная от МТС в библиотеке RecTools: <https://github.com/MobileTeleSystems/RecTools>



beeFormer: Bridging the Gap Between Semantic and Interaction Similarity in Recommender Systems

Vojtěch Vančura

vancurv@fit.cvut.cz

Faculty of Information Technology,
Czech Technical University in Prague

Prague, Czech Republic

Recombee

Prague, Czech Republic

Pavel Kordík

pavel.kordik@fit.cvut.cz

Faculty of Information Technology,
Czech Technical University in Prague

Prague, Czech Republic

Recombee

Prague, Czech Republic

Milan Straka

straka@ufal.mff.cuni.cz

Faculty of Mathematics and Physics,
Charles University

Prague, Czech Republic

beeFormer

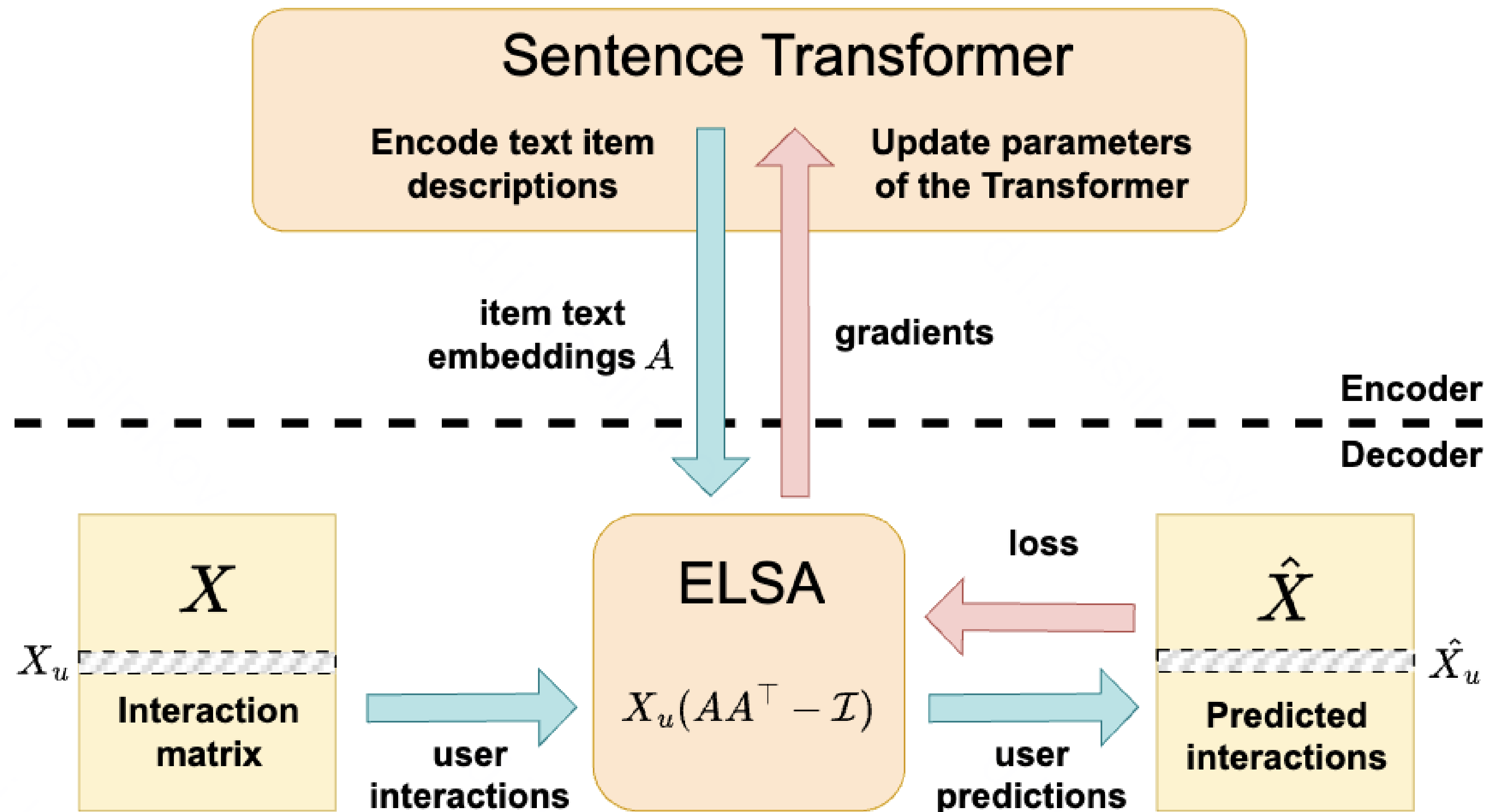


Figure 1: Training with the beeFormer framework: A sentence Transformer model (up) act as a an encoder to generate item embeddings represented by the matrix A . Then ELSA act as a decoder (down) in a training step on interactions to obtain gradients used to optimize the Transformer model.

3 TRAINING PROCEDURE

We follow notation from [47]: assume a set of users $\mathbb{U} = \{u_1, u_2, \dots, u_U\}$, a set of items $\mathbb{I} = \{i_1, i_2, \dots, i_I\}$, and a set of token sequences representing corresponding items $\mathbb{T} = \{t_1, t_2, \dots, t_I\}$. Let $X \in \{0, 1\}^{|\mathbb{U}| \times |\mathbb{I}|}$ be a user-item interaction matrix: $X_{a,b} = 1$ if the user u_a interacted with item i_b , and $X_{a,b} = 0$ otherwise. Assume that M_a is a *column* vector corresponding to the a -th row of matrix M . Let $\text{norm}(M)$ be a function that L^2 -normalizes each row of a matrix, so $\text{norm}(M)_a = M_a / \|M_a\|$. Finally, let $g(\bullet, \theta_g) : \mathbb{T} \rightarrow \mathbb{R}^{I \times d}$ be a Transformer-based neural network with parameters θ_g .

The training procedure starts with generating latent representations of items – the matrix A :

$$A = g(\mathbb{T}, \theta_g). \quad (1)$$

Then, we can compute our loss [47] as:

$$L = \left\| \text{norm}(X_u) - \text{norm}(X_u(AA^\top - \mathcal{I})) \right\|_F^2. \quad (2)$$

Finally, we compute the gradients of L with respect to A , and then the gradients for θ_g are computed using the chain rule. However,

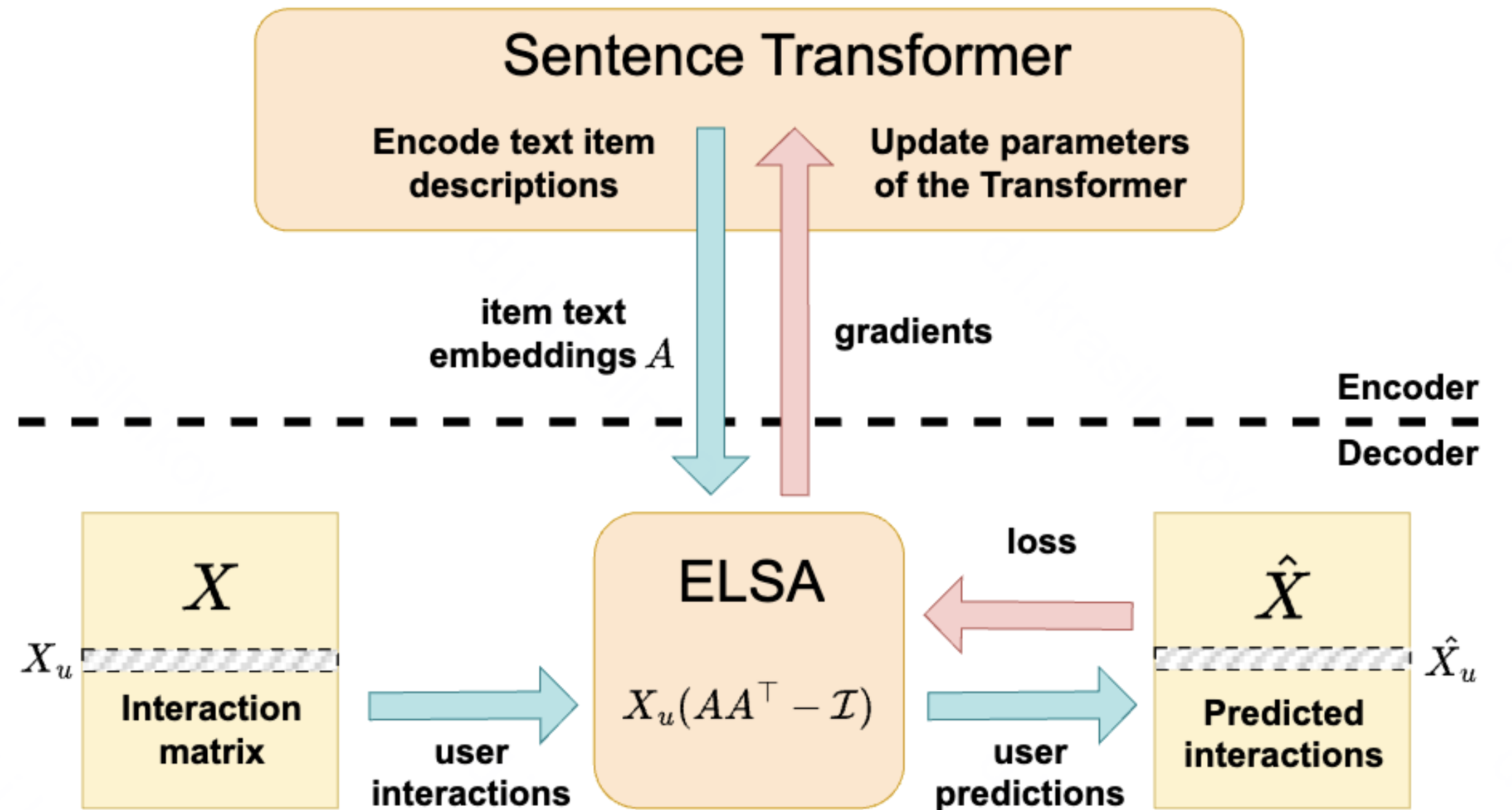


Table 3: Results for cold-start scenario in item-split setup.

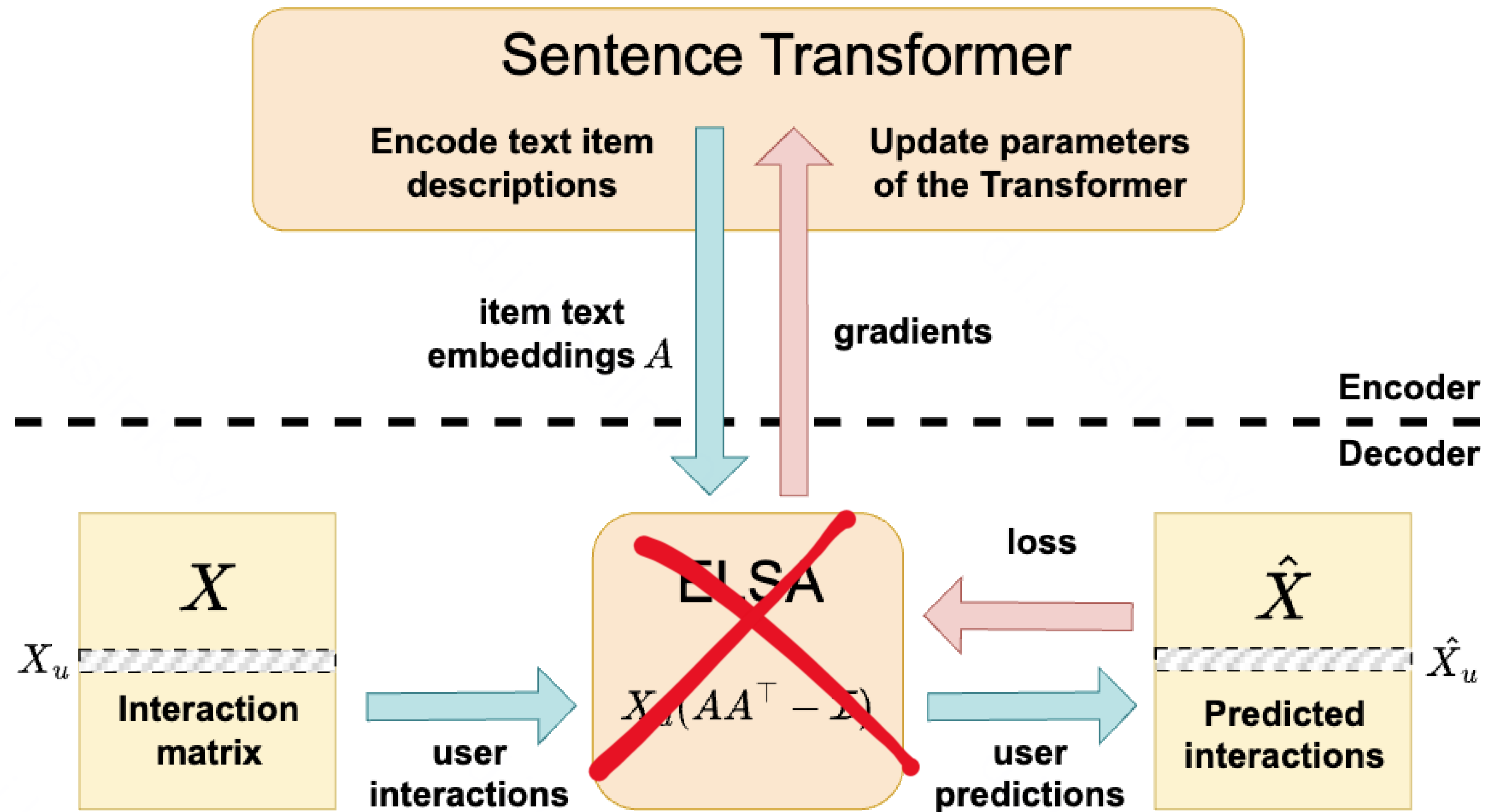
Names of our models trained with beeFormer are in italics, the best-performing models are represented in bold, and the best baseline for each scenario is underlined. The standard error of all values is below 0.0001 (evaluated via bootstrap resampling).

Dataset Method	Sentence Transformer	R@20	R@50	N@100
GB10K	<i>Llama-goodbooks-mpnet</i>	0.2505	0.3839	0.3747
CBF	<i>Llama-goodlens-mpnet</i>	0.2710	0.4218	0.4066
	all-mpnet-base-v2	0.2078	0.3221	<u>0.3195</u>
GB10K	nomic-embed-text-v1.5	<u>0.2154</u>	<u>0.3317</u>	0.3193
Heater	bge-m3	0.2052	0.3113	0.3099
	<i>Llama-movielens-mpnet</i>	0.2060	0.3161	0.3196
ML20M	<i>Llama-movielens-mpnet</i>	0.4291	0.6108	0.4054
CBF	<i>Llama-goodlens-mpnet</i>	0.4630	0.6152	0.4066
	all-mpnet-base-v2	<u>0.3114</u>	<u>0.4331</u>	<u>0.3407</u>
ML20M	nomic-embed-text-v1.5	0.3049	0.4285	0.3270
Heater	bge-m3	0.2847	0.3932	0.3161
	<i>Llama-goodbooks-mpnet</i>	0.3204	0.4669	0.3381

Table 4: Results for time-split setup on the Amazon Books dataset.

Names of our models trained with beeFormer are in italics, the best-performing models are represented in bold, and the best baseline for each scenario is underlined. The standard error of all values is below 0.00005 (evaluated via bootstrap resampling).

Scenario Method	Model	R@20	R@50	N@100
	all-mpnet-base-v2	0.0218	0.0336	0.0193
	nomic-embed-text-v1.5	0.0387	<u>0.0560</u>	<u>0.0320</u>
zero-shot	bge-m3	<u>0.0398</u>	0.0546	0.0313
CBF	<i>Llama-goodbooks-mpnet</i>	0.0649	0.0931	0.0515
	<i>Llama-goodlens-mpnet</i>	0.0617	0.0891	0.0492
supervised	KNN	0.0370	0.0562	0.0303
CF	ALS MF	0.0344	0.0580	0.0313
	ELSA	0.0367	0.0628	0.0346
	SANSA	<u>0.0421</u>	<u>0.0678</u>	<u>0.0362</u>
CBF	<i>Llama-amazbooks-mpnet</i>	0.0706	0.1045	0.0571



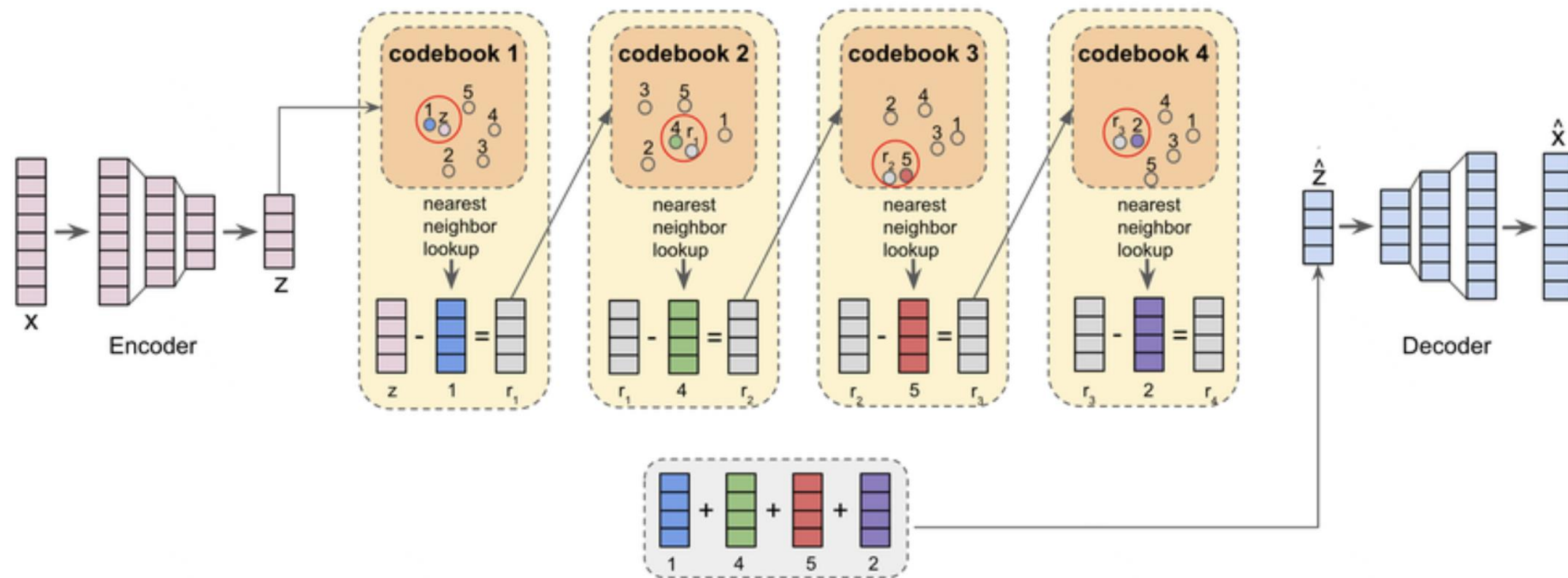
BETTER GENERALIZATION WITH SEMANTIC IDs: A CASE STUDY IN RANKING FOR RECOMMENDATIONS

Anima Singh^{*1}, Trung Vu^{*2}, Nikhil Mehta^{*1}, Raghunandan Keshavan², Maheswaran Sathiamoorthy¹,
Yilin Zheng², Lichan Hong¹, Lukasz Heldt², Li Wei², Devansh Tandon², Ed H. Chi¹, Xinyang Yi¹

¹Google DeepMind

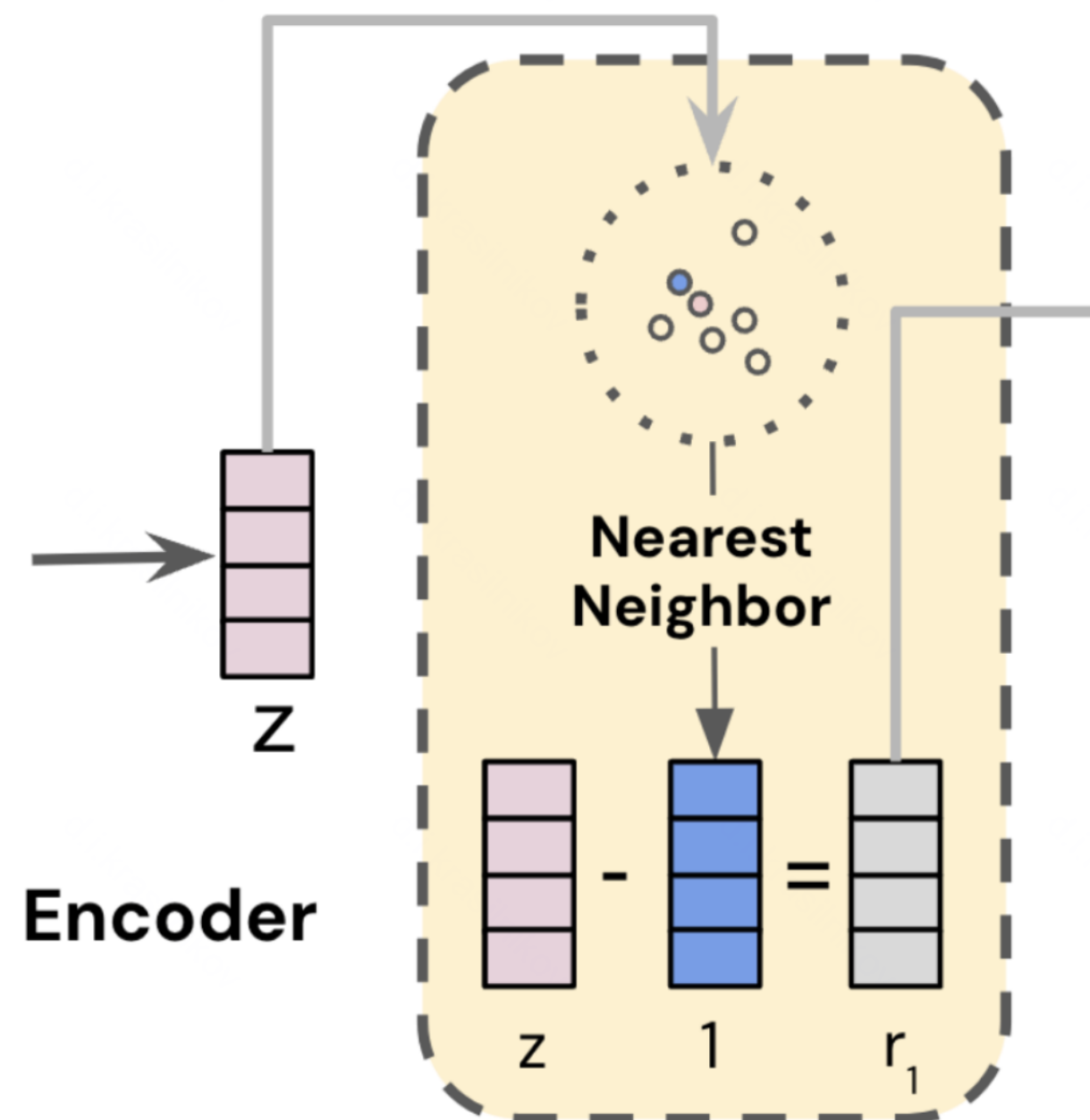
²Google

RQ-VAE - алгоритм работы



Z - эмбеддинг айтема полученный из энкодера
 $\hat{Z}$ - кодовое представление айтема
 $\hat{X}$ - восстановленное изначально представление

RQ-VAE - алгоритм работы

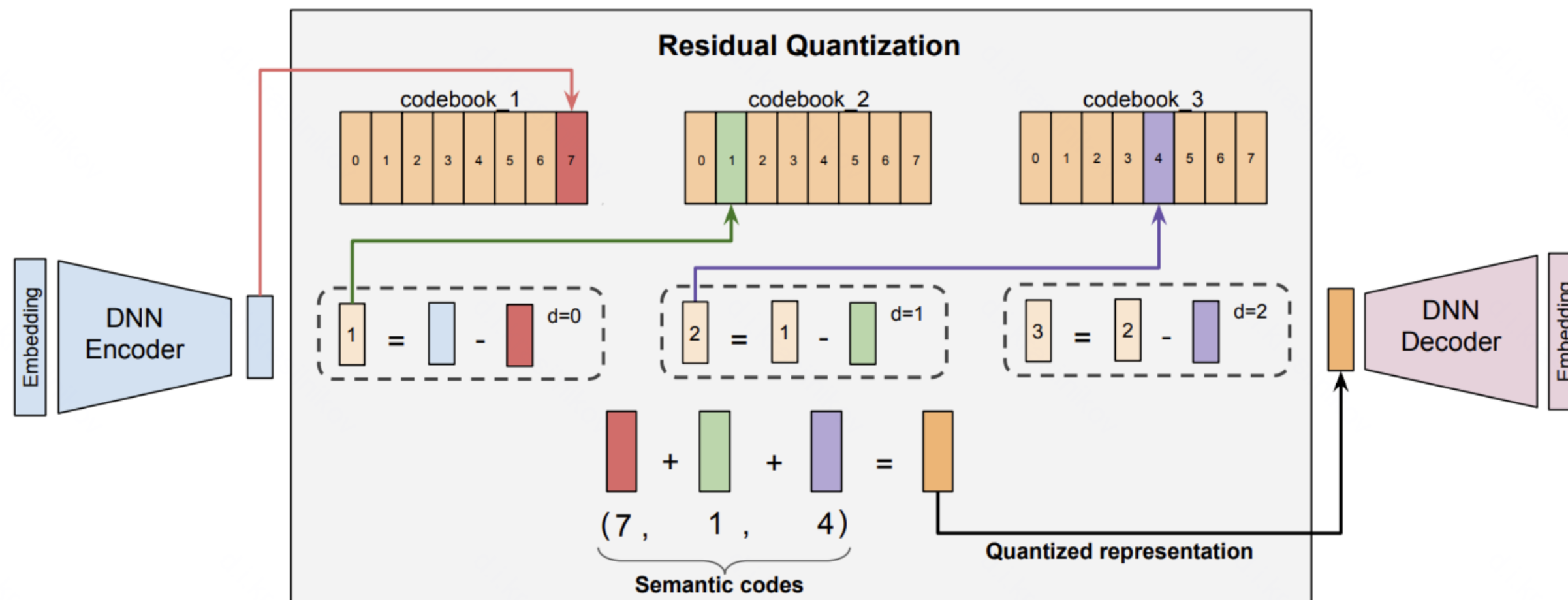


1. Энкодер: $z = E(x)$ — преобразует текст x в вектор z .
2. Квантование: $c_0 = \text{argmin}_k \|r_0 - e_k\|$, где $r_0 = z$.
3. Остаток: $r_1 = r_0 - e_{\{c_0\}}$ — передается на следующий уровень.

Semantic ID: $(c_0, c_1, \dots, c_{\{m-1\}})$ — финальная последовательность кодов.

Декодирование: $\hat{z} = \sum_{d=0}^{m-1} e_{\{c_d\}}$ — приближает исходный z .

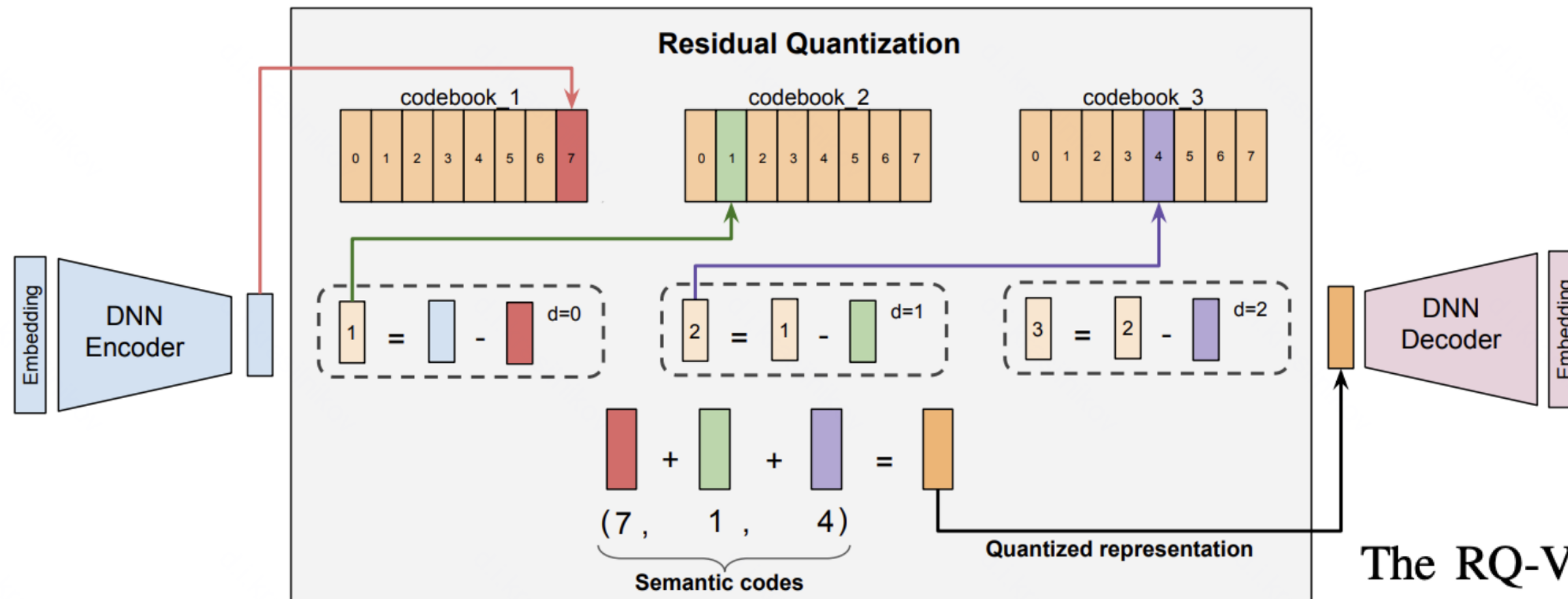
RQ-VAE



3. **Semantic ID:** $(c_0, c_1, \dots, c_{m-1})$ — финальная последовательность кодов.

4. Декодирование: $\hat{z} = e_{c_0} + e_{c_1} + e_{c_{m-1}}$ — приближает исходный z .

RQ-VAE



The RQ-VAE loss is defined as $\mathcal{L}(\mathbf{x}) := \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{rqvae}}$,

$$\mathcal{L}_{\text{recon}} := \|\mathbf{x} - \hat{\mathbf{x}}\|^2$$

$\mathcal{L}_{\text{rqvae}} := \sum_{d=0}^{m-1} \|\text{sg}[\mathbf{r}_i] - \mathbf{e}_{c_i}\|^2 + \beta \|\mathbf{r}_i - \text{sg}[\mathbf{e}_{c_i}]\|^2$. Here $\hat{\mathbf{x}}$ is the output of the decoder, and sg is the stop-gradient operation [35]. This loss jointly trains the encoder, decoder, and the codebook.

Использовали $\beta = 0.25$.

Альтернативные методы построения ID



- **Locality Sensitive Hashing (LSH):**

- Простой метод для создания семантических ID.
- В проведённом исследовании (раздел 4.2) показал худшие результаты по сравнению с RQ-VAE.

- **Иерархический k-means кластеринг:**

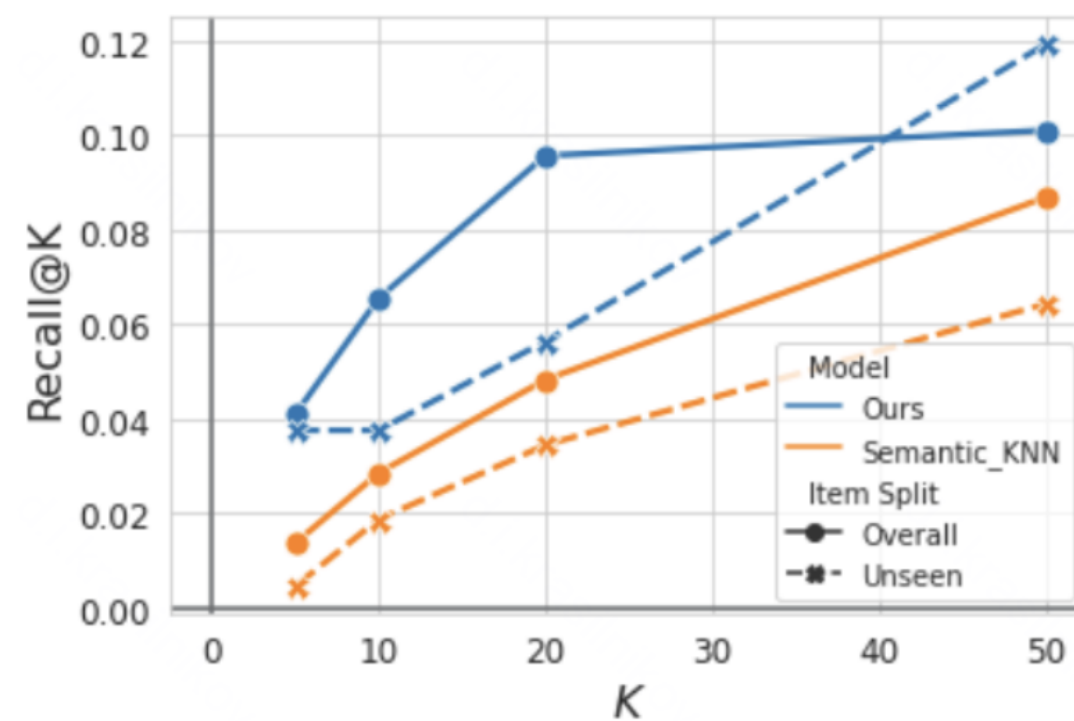
- Позволяет организовать ID иерархически.
- Теряет семантическое значение между различными кластерами, что снижает его эффективность для данной задачи.

- **VQ-VAE:**

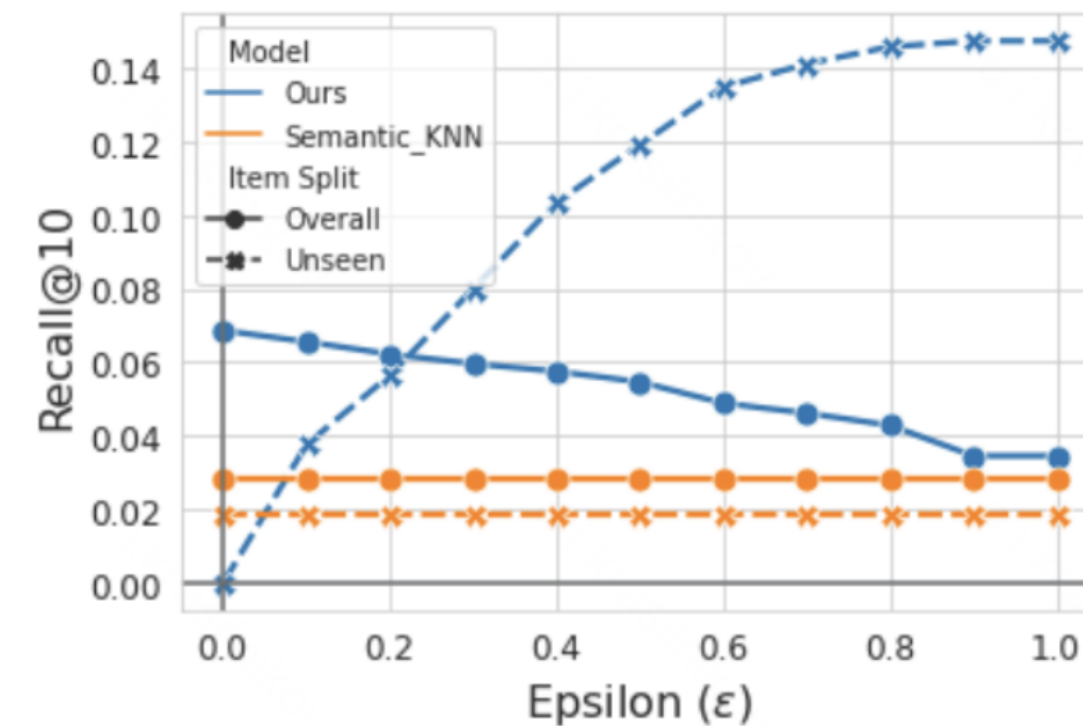
- Работает схоже с RQ-VAE для генерации кандидатов на этапе поиска.
- Не обеспечивает иерархическую структуру ID, что ограничивает его функциональность по сравнению с RQ-VAE.

Table 2: Ablation study for different ID generation techniques for generative retrieval. We show that RQ-VAE Semantic ID (SID) perform significantly better compared to hashing SIDs and Random IDs.

Methods	Sports and Outdoors				Beauty				Toys and Games			
	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10
Random ID	0.007	0.005	0.0116	0.0063	0.0296	0.0205	0.0434	0.0250	0.0362	0.0270	0.0448	0.0298
LSH SID	0.0215	0.0146	0.0321	0.0180	0.0379	0.0259	0.0533	0.0309	0.0412	0.0299	0.0566	0.0349
RQ-VAE SID	0.0264	0.0181	0.0400	0.0225	0.0454	0.0321	0.0648	0.0384	0.0521	0.0371	0.0712	0.0432



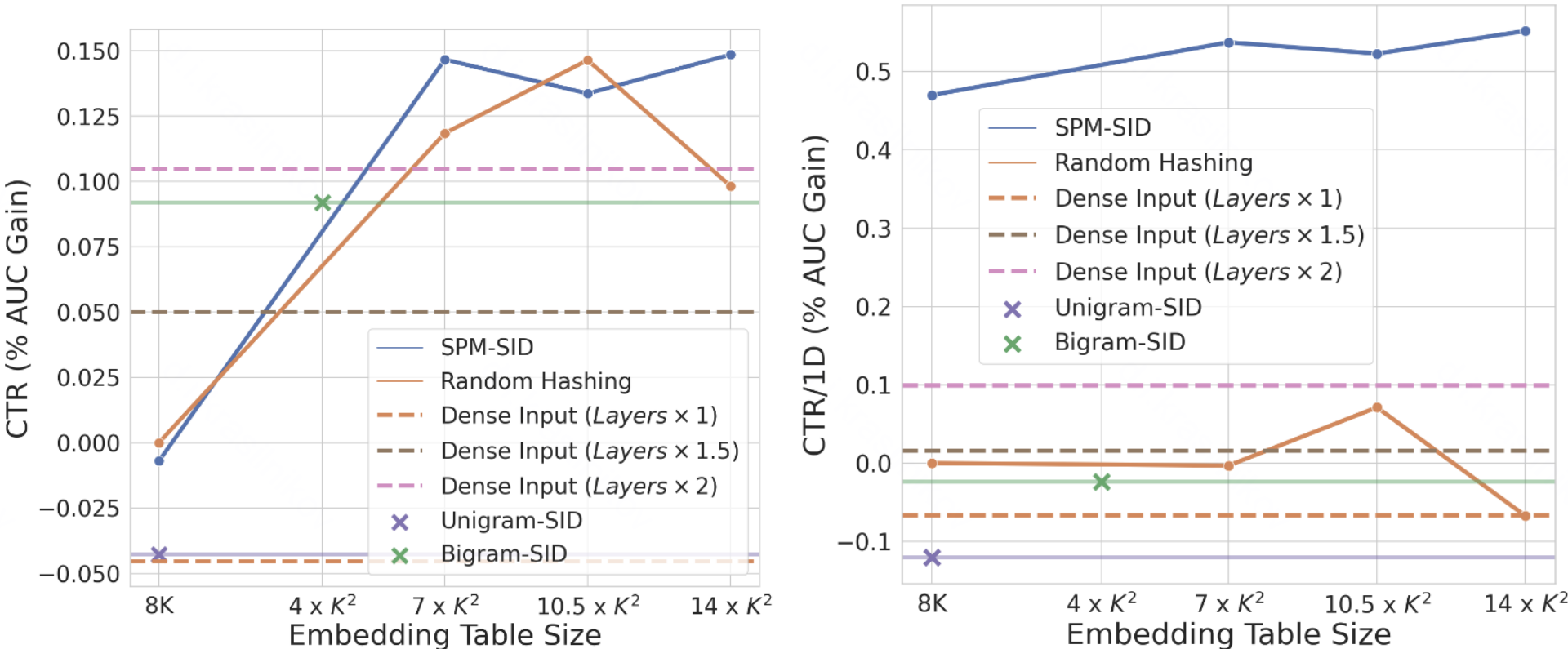
(a) Recall@K vs. K, ($\epsilon = 0.1$).



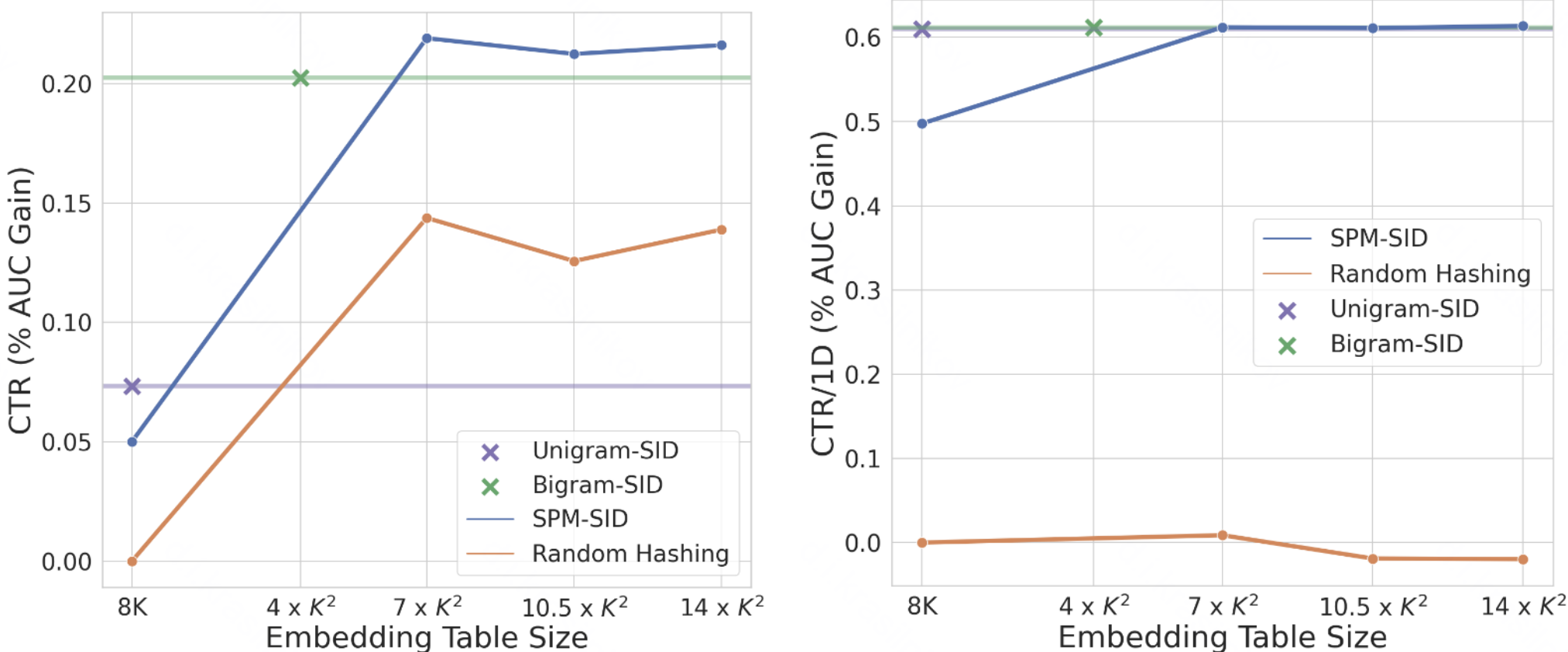
(b) Recall@10 vs. ϵ .

Figure 5: Performance in the cold-start retrieval setting.

Результаты

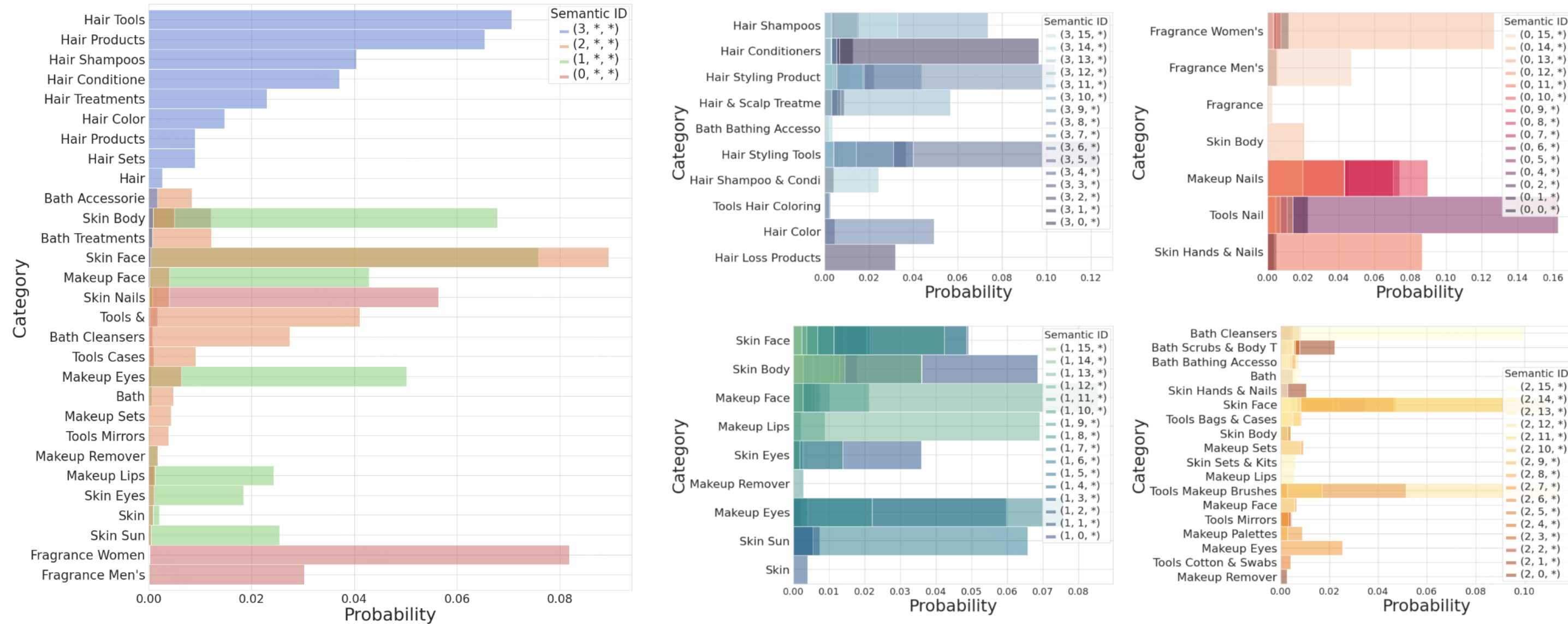


(a) Overall CTR AUC
Figure 2: Percentage improvement in CTR AUC metric when user history is not used as a input feature. Improvement is relative to Random Hashing baseline with 8K embedding table size.



(a) Overall CTR AUC
Figure 3: Percentage improvement in CTR AUC metric when user history is used as a input feature. Improvement is relative to Random Hashing baseline with 8K embedding table size.

Semantic IDs



(a) The ground-truth category distribution for all the items in the dataset colored by the value of the first codeword c_1 .

(b) The category distributions for items having the Semantic ID as $(c_1, *, *)$, where $c_1 \in \{1, 2, 3, 4\}$. The categories are color-coded based on the second semantic token c_2 .

Figure 4: Qualitative study of RQ-VAE Semantic IDs (c_1, c_2, c_3, c_4) on the Amazon Beauty dataset. We show that the ground-truth categories are distributed across different Semantic tokens. Moreover, the RQVAE semantic IDs form a hierarchy of items, where the first semantic token (c_1) corresponds to coarse-level category, while second/third semantic token (c_2/c_3) correspond to fine-grained categories.

Спасибо за
внимание